

Vysoká škola ekonomická v Praze
Fakulta informatiky a statistiky



Návrh a testování prostředí pro AI agenty v softwarovém vývoji

BAKALÁŘSKÁ PRÁCE

Studijní program: Aplikovaná informatika

Autor: Thanh An Nguyen

Vedoucí práce: Ing. Jiří Korčák

Praha, květen 2026

Prohlášení

Prohlašuji, že jsem bakalářskou práci zpracoval samostatně a že jsem uvedl všechny použité prameny a literaturu, ze kterých jsem čerpal.

Potvrzuji, že při přípravě předložené práce **byly použity** nástroje umělé inteligence. Konkrétně byly použity těmito způsoby:

- úprava a rozšíření textových částí,
- vyhledávání a shrnutí literatury,
- diagnostická analýza výsledků experimentu,
- generování částí zdrojových kódů měřicí infrastruktury.

Každé použití některého ze způsobů je samostatně dokumentováno v příloze A, kde je uvedeno podrobnější vysvětlení, který nástroj byl ve které části práce konkrétně použit, a je zvýrazněn vlastní přínos.

Potvrzuji, že:

- má práce je originální a byla zpracována v souladu s etickými standardy, zejména s Etickým kodexem Vysoké školy ekonomické v Praze; že jsou v práci citovány všechny použité informační zdroje a že generovaný obsah byl mnou ověřen;
- přijímám plnou odpovědnost za celý obsah závěrečné práce.

V Praze dne 6. května 2026

Thanh An Nguyen

Poděkování

Děkuji vedoucímu práce Ing. Jiřímu Korčákovi za odborné vedení, trpělivost a věcné připomínky, které práci v průběhu jejího vzniku významně posunuly. Poděkování patří také mé rodině a blízkým za podporu během celého studia.

Abstrakt

AI coding agenti dokáží samostatně generovat kód, testy i vývojové artefakty, ale jejich praktická použitelnost nezávisí jen na tom, zda výsledný program projde testy. Pro nasazení ve vývoji je důležité také to, jak agent pracoval, zda zanechal auditovatelnou stopu a jakou kvalitu má výsledný kód. Současné benchmarky tyto dimenze typicky nezachycují společně a chybí postup, jak podle nich systematicky navrhovat instrukce. Práce proto navrhuje sadu metrik pokrývající proces, kvalitu produktu a zdroje a ukazuje, jak ji použít k iterativnímu návrhu instrukcí. Postup demonstruje na případové studii systému upomínek faktur: agent opakovaně implementuje stejnou specifikaci, výsledky jsou vyhodnoceny navrženými metrikami a instrukce jsou mezi běhy upravovány podle diagnostiky selhání. Následné ablace zkoumají, které složky instrukcí mají měřitelný přínos. Výsledky ukazují, že metriky dokáží odlišit funkčně úspěšný výstup od výstupu vzniklého slabým nebo neauditovatelným procesem. Iterativní úpravy instrukcí vedly ke zlepšení, ale ne monotónně: některé změny způsobily regresi a stejné instrukce vedly v různých bězích k odlišnému dodržení pracovního postupu. Opakovaným vzorcem úspěšných úprav byl posun od obecného pravidla ke konkrétnímu příkazu a nakonec k verifikačnímu kroku. Ablace ukázaly, že verifikační kroky nejsou redundantní. Odebrání části kódových konvencí deterministické metriky téměř nezhoršilo, ale zhoršilo designovou kvalitu hodnocenou LLM-as-judge. Přínosem práce je demonstrace proveditelnosti metrického rámce a iterativního postupu návrhu instrukcí. Konkrétní naměřené hodnoty platí pro daný model, nástroj a projekt; přenositelný je postup a sada metrik. Otevřenou otázkou zůstává, zda vzorec operacionalizace platí i mimo programování.

Klíčová slova

AI coding agent, AGENTS.md, metriky kvality software, iterativní návrh, ablační studie

Abstract

AI coding agents can generate code, tests, and development artifacts autonomously, but their practical usefulness depends on more than whether the resulting program passes tests. In software development, it also matters how the agent worked, whether it left an auditable trace, and what quality the resulting code has. Existing benchmarks typically do not capture these dimensions together, and there is no established process for designing instructions against such measurements. This thesis proposes a metric suite covering process, product quality, and resources, and shows how it can guide iterative instruction design. The process

is demonstrated on a case study of a dunning system: the agent repeatedly implements the same specification, each run is evaluated using the proposed metrics, and the instructions are revised according to a diagnosis of observed failures. Subsequent ablations examine which parts of the instructions have a measurable contribution. The results show that the metrics can distinguish a functionally successful output from one produced by a weak or non-auditable process. Iterative instruction changes improved the agent’s behavior, but not monotonically: some changes caused regressions, and identical instructions led to different levels of process adherence across runs. A recurring pattern in successful changes was a shift from a general rule to a specific command and then to a verification step. Ablations showed that verification steps were not redundant. Removing part of the code-convention section had little effect on automated metrics, but reduced design quality as assessed by an LLM-as-judge. The contribution is a feasibility demonstration of the metric framework and the iterative instruction-design process. The specific measured values are bound to the model, tool, and project used in the case study; what transfers is the process and the metric suite. An open question is whether the observed operationalization pattern also holds outside programming.

Keywords

AI coding agent, AGENTS.md, software quality metrics, iterative design, ablation study

Obsah

Úvod	11
1 Vymezení problému a cílů práce	12
1.1 Motivace	12
1.2 Cíle práce	13
1.3 Rozsah práce	13
2 Teoretická východiska	14
2.1 Kvalita software a její měření	14
2.1.1 Procesní kvalita	15
2.1.2 Produktová kvalita	16
2.1.3 Zdrojová dimenze	18
2.2 AI coding agenti	19
2.2.1 Základní pojmy	19
2.2.2 Jak agenti mění softwarové inženýrství	20
2.2.3 Hodnocení schopností agentů	21
2.2.4 Instrukce jako nezávislá proměnná	22
3 Metodika	25
3.1 Výzkumný přístup	25
3.1.1 Volba výzkumné strategie	25
3.1.2 Příklad a jeho prostředí	26
3.1.3 Fáze a iterativní cyklus	29
3.2 Sada metrik	30
3.2.1 Procesní metriky (P1–P8)	31
3.2.2 Produktové metriky (Q1–Q8)	32
3.2.3 Metriky zdrojů (E1–E3)	33
3.2.4 Aplikace LLM-as-judge	34
3.3 Experimentální design	35
3.3.1 Fixní proměnné	35
3.3.2 Záznam a vyhodnocení běhu	36
3.3.3 Diagnostika a úprava instrukcí	37
3.4 Přehledová tabulka	38
3.5 Omezení a validita	38
4 Praktická část	40
4.1 Konstrukce baseline instrukcí	40
4.2 Pilotní fáze	42
4.2.1 Pilot-r1: baseline	42
4.2.2 Pilot-r2	43

4.2.3	Pilot-r3	45
4.2.4	Pilot-r4	46
4.2.5	Pilot-r5: verifikace kontraktu	48
4.3	Ablace	49
4.3.1	Výběr složek pro ablaci	49
4.3.2	Ablace A: bez verifikačních kroků	51
4.3.3	Ablace B: bez Package Quality	52
4.4	Souhrnné výsledky	53
5	Vyhodnocení a diskuse	56
5.1	Interpretace výsledků	56
5.1.1	Cíl 1: Sada metrik	56
5.1.2	Cíl 2: Iterativní postup	60
5.1.3	Cíl 3: Ablace	62
5.2	Porovnání s literaturou	63
5.2.1	Srovnání metrického rámce	63
5.2.2	Vliv instrukcí na chování agenta	63
5.3	Omezení a jejich dopad	64
5.4	Doporučení pro praxi	66
5.5	Náměty pro další výzkum	67
	Závěr	70
	Literatura	71
	A Využití nástrojů umělé inteligence	78

Seznam obrázků

3.1	Stavový automat systému upomínek	27
3.2	Iterativní cyklus: čtyři kroky jedné iterace a smyčka opakování až do splnění exit kritérií.	30
4.1	Baseline AGENTS.md (pilot-r1): kompletní znění instrukcí použitých jako výchozí bod pilotních iterací.	41
4.2	Vizuální diff změn AGENTS.md mezi pilot-r1 a pilot-r2	43
4.3	Vizuální diff změn AGENTS.md mezi pilot-r2 a pilot-r3	45
4.4	Vizuální diff změn AGENTS.md mezi pilot-r3 a pilot-r4	46
4.5	Vizuální diff změn AGENTS.md mezi pilot-r3 a pilot-r5	48
4.6	Změny AGENTS.md pro Ablaci A	50
4.7	Změny AGENTS.md pro Ablaci B: odebrána celá sekce Package Quality (konvence, strict TypeScript, JSDoc, čisté API).	50
5.1	Míra splnění exit kritérií procesní metriky (P1–P8) napříč devíti běhy (deterministické P1–P5: pass; judge-based P6–P8: $\geq 2/3$). P2 a P5 byly splněny nejméně často (3/9), P6 a P7 ve všech bžích. Deterministické položky (P1–P5) vykazují vyšší variabilitu než kvalitativní (P6–P8).	57
5.2	Míra splnění operačních prahů produktové metriky (Q1–Q8) napříč devíti běhy ($Q2 \geq 37$, $Q3 \geq 70\%$, $Q4 \geq 24$, $Q5 \leq 1$, $Q7 \leq 1$; viz sekce 3.1.3). Striktní exit z tabulky 3.10 ($Q2 = 42/42$, $Q5 = 0$, $Q7 = 0$ porušení) nesplnil žádný běh. Q1 a Q6 byly splněny ve všech bžích; Q3 a Q5 nejméně často. Q3 počítáno ze 7 běhů (2 bez dat). Metriky efektivity (E1–E3) nemají exit kritérium a nejsou zahrnuty.	59
5.3	Cena běhu vs. počet splněných deterministických kritérií	60
5.4	Spektrum operacionalizace instrukcí. Tři nezávislá selhání (P2, P3, Q8) prošla stejnými fázemi: pravidlo (agent ignoruje) → příkaz (agent obchází) → verifikační krok (agent dodržuje).	61

Seznam tabulek

3.1	Procesní compliance metriky P1–P5	31
3.2	Judge-based procesní metriky P6–P8	32
3.3	Metriky funkční korektnosti Q1–Q2	32
3.4	Metriky kvality testů Q3–Q4	33
3.5	Metriky kvality kódu Q5–Q8	33
3.6	Metriky zdrojů E1–E3	34
3.7	Záznamy pořízené pro jeden běh a jednu iteraci	36
3.8	Diagnostické rámce a otázky, které kladou	37
3.9	Mapování klasifikace příčiny na diagnostický rámec a typ opravy	37
3.10	Sada metrik: kód, měřená vlastnost, nástroj a exit kritérium	38
4.1	Mapování sekcí baseline AGENTS.md na metriky	40
4.2	Pilot-r1: naměřené metriky	42
4.3	Pilot-r2: metriky se změnou oproti r1	44
4.4	Pilot-r3: metriky se změnou oproti r2	45
4.5	Pilot-r4: metriky se změnou oproti r3	47
4.6	Pilot-r5: metriky se změnou oproti r3	48
4.7	Ablace A: srovnání s r3 (bez verifikačních kroků)	51
4.8	Ablace B: srovnání s r3 (bez sekce Package Quality)	52
4.9	Souhrnné výsledky všech běhů (pilot + ablace)	54

Seznam použitých zkratk

E1 vstup/výstup/cache tokeny

E2 trvání

E3 počet kompakcí

P1 issues before code

P2 branch per issue

P3 test-first commits

P4 PRs linked to issues

P5 no existing test modifications

P6 commit message quality

P7 issue description quality

P8 PR description quality

Q1 API contract match

Q2 referenční test pass rate

Q3 mutation score

Q4 AC coverage

Q5 lint warnings

Q6 typecheck errors

Q7 složitost kódu

Q8 design quality

Úvod

AI coding agenti dokáží autonomně implementovat funkcionalitu, psát testy, spravovat verzovací historii a komunikovat prostřednictvím issues a pull requestů. S rostoucí autonomií těchto nástrojů roste i potřeba systematicky hodnotit kvalitu jejich práce. Současné benchmarky hodnotí izolované aspekty práce agenta, především funkční korektnost. Holistický pohled, který by zachytil proces, kvalitu produktu a zdroje společně, však chybí. Přitom i funkčně správné řešení může být v praxi odmítnuto kvůli nedostatkům v procesu nebo udržitelnosti. Praktik, který vidí jen pass/fail výsledek, tak může přecenit řešení, které by ve skutečném vývoji neprošlo code review nebo by se obtížně rozvíjelo.

Kvalitu práce agenta výrazně ovlivňují instrukce, které dostane. V praxi se etabloují soubory s instrukcemi jako `AGENTS.md` nebo `CLAUDE.md`, které definují pracovní postup, konvence a omezení projektu. Tyto soubory se stávají standardním rozhraním mezi člověkem a agentem nejen v softwarovém vývoji, ale i v univerzálních agentních nástrojích. Jak instrukce systematicky navrhovat a jak měřit, zda agent dodržuje požadované praktiky, zůstává otevřenou otázkou.

Tato práce navrhuje sadu metrik pokrývající proces, kvalitu produktu a zdroje spotřebované při práci agenta. Na případové studii necháváme agenta opakovaně implementovat systém upomínek faktur ze specifikace, výsledky měříme navrženými metrikami a na jejich základě upravujeme instrukce mezi iteracemi. Ablacemi dále zkoumáme, které složky instrukcí přispívají k měřenému chování a které jsou redundantní.

Práce nejprve vymezuje problém a cíle, poté shrnuje teoretická východiska z oblasti softwarového inženýrství, AI agentů a měření kvality. Na jejich základě navrhuje metodiku, kterou ověřuje na případové studii. Kapitola Vyhodnocení a diskuse interpretuje výsledky ve vztahu k cílům a porovnává je s existujícím výzkumem.

1. Vymezení problému a cílů práce

1.1 Motivace

AI coding agenti, tedy autonomní systémy schopné samostatně psát kód, spravovat verzovací historii a interagovat s vývojovým prostředím, se v posledních letech staly rozšířeným nástrojem každodenního vývoje softwaru. Slibují výrazné zrychlení práce: implementace, která vývojáři zabere hodiny, je teoreticky proveditelná v řádu minut. Empirická data ale ukazují, že tento příslib se nenaplnuje automaticky. Randomizovaná studie METR (METR, 2025) na 246 úlohách doložila, že vývojáři s AI nástroji byli ve skutečnosti o 19 % pomalejší.

Současné benchmarky jako SWE-bench (Jimenez et al., 2024) hodnotí izolované aspekty práce agenta, především funkční korektnost. Tři empirické studie ale shodně ukazují, že samotný pass/fail výsledek pro praktickou použitelnost nestačí. METR (METR, 2026)¹ doložila, že polovina patchů projitých automatickými testy by nebyla přijata při code review. Li et al. (B. Li et al., 2026) na 1 210 sloučených pull requestech zjistili kvalitativní nedostatky i v přijatém kódu. Ehsani et al. (Ehsani et al., 2026) pak z 33 tisíc agentních pull requestů identifikovali, že vedle funkčních chyb tvoří významný podíl odmítnutí i procesní selhání. Každá studie pojmenovává jinou stranu téhož problému: pass/fail neměří, zda by řešení obstálo v code review, zda je kód udržovatelný a zda vznikl auditovatelným postupem. Holistický pohled, který by tyto dimenze zachytil společně, dosud chybí.

Výsledky agenta nejsou dány jen schopnostmi modelu samotného; lze je ovlivnit dvěma způsoby: trénováním modelu nebo instrukcemi v kontextovém okně. S rostoucí velikostí kontextových oken a zkracujícími se cykly mezi generacemi modelů se instrukce stávají praktičtější alternativou: jsou dostupné komukoliv, iterativně upravitelné a přenositelné mezi modely. Fine-tuning naproti tomu vyžaduje trénovací data, výpočetní kapacitu a investici vázanou na konkrétní verzi modelu. Shin et al. (Shin et al., 2025) ukazují, že iterativní zpřesňování instrukcí dosahuje srovnatelných výsledků s fine-tuned modely na úlohách s kódem.

Empirie o účinnosti instrukcí je ale nejednoznačná. Lulla et al. (Lulla et al., 2026) zjistili, že přítomnost souboru s instrukcemi je spojena se zkrácením doby běhu o 28,6 %, zatímco Gloaguen et al. (Gloaguen et al., 2026) upozorňují, že generické instrukce přinášejí marginální zlepšení při vyšších nákladech. Které složky instrukcí k měřenému chování skutečně přispívají a které jen zabírají kontextové okno, dosud nebylo izolováno. Jak instrukce systematicky navrhovat a jak měřit, zda agent dodržuje požadované chování, zůstává proto otevřenou otázkou.

¹Jiná studie téže organizace než (METR, 2025); zde analýza SWE-bench pull requestů, ne randomizovaný experiment.

1.2 Cíle práce

1. Na základě analýzy existujících standardů kvality softwaru a současných benchmarků pro AI agenty navrhnout sadu metrik pokrývající proces, kvalitu produktu a zdroje, a tím zachytit dimenze, které stávající benchmarky neměří.
2. Na případové studii demonstrovat iterativní postup návrhu instrukcí řízený těmito metrikami a vyhodnotit, zda a jak vede k měřitelným změnám v chování agenta podle navržených metrik.
3. Z instrukcí vytvořených v cíli 2 prozkoumat ablacemi, které složky přispívají k měřenému chování agenta a které jsou redundantní.

1.3 Rozsah práce

Cíle ověřujeme na případové studii systému upomínek faktur. Existující studie hodnotí agenty na velkém počtu krátkých úloh; naše práce volí opačný přístup: jeden projekt do hloubky, s opakovanými běhy a podrobnou analýzou každého z nich.

Práce se nezabývá trénováním modelů, protože cílem je zkoumat vliv instrukcí, ne schopnosti konkrétního modelu. Nesrovnává různé modely ani programovací jazyky, protože experimentální design se snaží držet ostatní podmínky co nejstabilnější a měnit především instrukce. Neporovnává agenta s lidským vývojářem, protože měří zlepšení mezi iteracemi, ne absolutní výkon. Navržená sada metrik a iterativní postup jsou koncipovány jako přenositelný rámec; konkrétní instrukce a naměřené hodnoty platí pro tuto případovou studii a slouží jako demonstrace proveditelnosti, nikoli jako obecně platný výsledek pro všechny modely a projekty. Zdůvodnění volby projektu a podrobnosti experimentálního designu popisuje kapitola 3.

2. Teoretická východiska

Tato kapitola poskytuje teoretický základ pro metrický rámec a postup případové studie zaměřené na návrh a vyhodnocení instrukcí. První sekce se věnuje kvalitě software z pohledu softwarového inženýrství: proč je její zajištění obtížné, jaké dimenze má, jakými praktikami se zajišťuje a jak se měří. Druhá sekce zavádí AI coding agenty: co jsou, jak se s nimi mění softwarové inženýrství, jak se jejich výstupy dnes hodnotí a jak se jejich chování řídí instrukcemi.

2.1 Kvalita software a její měření

Software je už ze své povahy složitý, a tato složitost má dva odlišné zdroje. Část pochází z povahy řešeného problému: domény, ve kterých software pracuje, jsou samy o sobě komplexní (mají mnoho stavů, výjimek a vzájemných závislostí), a tato složitost se nevyhnutelně přenáší do systému, který je má modelovat. Druhá část přichází z nástrojů a technologií, kterými software stavíme: programovací jazyky, frameworky, build systémy a knihovny vnášejí vlastní vrstvu složitosti, která s podstatou úlohy přímo nesouvisí. Brooks (Brooks, 1987) tyto dva zdroje pojmenoval jako *esenciální* a *akcidentální* složitost a argumentuje, že zatímco akcidentální vrstvu lze snižovat lepšími nástroji, esenciální zůstává a je třeba ji systematicky zvládat.

Aby vývojáři tuto složitost udrželi pod kontrolou, opírá se obor o soubor zavedených praktik (testování, revize kódu, správa verzí, systematická diagnostika chyb), které tvoří jeho metodickou páteř (IEEE Computer Society, 2024). Samotné dodržování praktik ale kvalitu nezaručuje, a to ze dvou důvodů. Software, který se reálně používá, musí být průběžně upravován, jinak přestává odpovídat svému účelu (Lehman, 1980), a s každou úpravou roste jeho vnitřní složitost. Postupy vhodné v rané fázi vývoje proto nemusí být dostatečné později. Praktiky se navíc vyvíjejí společně s oborem, takže to, co bylo považováno za standard před deseti lety, dnes nemusí stačit. Kvalitu je proto třeba průběžně ověřovat měřením, ne jen předpokládat z deklarace, že se nějaká praktika dodržuje.

Měření ale nemůže stát na jediné metrice ani na jediném typu evidence. Fenton a Bieman (Fenton & Bieman, 2014) rozlišují tři kategorie měřitelných entit:

- *proces*, tedy aktivity a postupy, kterými produkt vzniká,
- *produkt*, tedy výstupy vývoje (implementace, testy, dokumentace),
- *zdroje*, tedy vstupy nutné k vzniku produktu (čas, úsilí, lidé, výpočetní prostředky).

Každá entita nese jiný typ evidence o kvalitě: dodržení procesu nezaručuje funkční výsledek, funkčně správný produkt může vzniknout i chaotickým procesem, a náklady, za které řešení vzniklo, jsou na obou předchozích nezávislé. Ucelený obraz proto vzniká až jejich kombinací.

Následující tři podsekce rozebírají každou dimenzi zvlášť: její konceptuální vymezení a způsoby,

jimiž ji literatura operacionalizuje. Hloubka zpracování se mezi nimi liší. Procesní stránka stojí na rodině etablovaných praktik vývoje software (sekce 2.1.1), produktová má dnes zralý standardizovaný rámec (sekce 2.1.2) a zdrojová se zachycuje převážně přes proxy nákladů, úsilí a času (sekce 2.1.3).

2.1.1 Procesní kvalita

Procesní kvalitou se rozumí vlastnosti aktivit a postupů, kterými produkt vzniká. Fenton a Bieman (Fenton & Bieman, 2014) mezi procesní atributy řadí zejména pracnost, trvání, shodu s definovaným postupem a stabilitu výstupu, tedy vlastnosti, které lze pozorovat na chování vývojáře, ne na hotovém kódu.

Stejný kód může vzniknout různými způsoby. Některé jsou auditovatelné a snadno opakovatelné, jiné stojí na jednorázových zkratkách a jejich stopa je pro tým obtížně dohledatelná. Verzovaný vývoj (typicky systémem Git, doplněným o platformové funkce jako issues a pull requesty na GitHubu nebo GitLabu) přitom po sobě zanechává řadu pozorovatelných artefaktů: commity, větve, pull requesty a záznamy z CI.

Z těchto artefaktů lze průběh práce zpětně zrekonstruovat, a právě z nich, ne z deklarace o tom, jaký postup se údajně dodržuje, je třeba procesní kvalitu měřit. Jak velký rozdíl bývá mezi tím, co lidé o svém procesu říkají, a tím, co skutečně dělají, ukázali Beller et al. (Beller et al., 2019) v rozsáhlé field study sledující 2 443 vývojářů v IDE po dobu 2,5 roku: polovina z nich systematicky netestuje, test-driven development není rozšířený ani v komunitě, která se k němu hlásí, a čas skutečně strávený testováním je výrazně nižší než ten, který si vývojáři sami přisuzují.

Issue-driven vývoj a větvení.

Změna v projektu obvykle začíná *issue*, tedy záznamem v issue trackeru, který popisuje, co je třeba udělat a jak poznat, že je úkol hotov. Issue plní dvě role současně: dává implementaci jasné zadání ještě předtím, než vznikne kód, a zároveň zůstává trvalým záznamem, k němuž lze později dohledat související změny. Velkoplošná empirická studie Bissyandé et al. (Bissyandé et al., 2013) na stovkách tisíc projektů na GitHubu doložila, že issue trackery jsou v open source vývoji rozšířenou a aktivně používanou součástí projektové infrastruktury.

K issue se typicky váže vlastní vývojová větev, do které se implementace zapisuje. Práce na jedné změně tak zůstává izolovaná od ostatních, paralelní vývoj se vzájemně neruší a code review může posuzovat rozsah změny ohraničeně.

Atomicita a konvence commitů.

Granulární commity, kde každá změna odpovídá jednomu logickému kroku, usnadňují zpětnou analýzu chyb i code review (Humble & Farley, 2010). Kromě atomicity se v praxi rozšířila i konvence formátu commit zprávy: prefix v podobě **feat:**, **test:** nebo **fix:** signalizuje typ změny ještě před přečtením celé zprávy a zkracuje čas potřebný k orientaci v historii. Atomicita a konzistentní formát se ukazují jako nutná podmínka toho, aby commity nesly užitečnou informaci pro lidského i automatizovaného čtenáře.

Test-driven development a stabilita testů.

Testování v průběhu implementace může mít různé pořadí, a zvolené pořadí mění roli, kterou test ve vývoji hraje. *Test-driven development* obrací nejběžnější *test-after* pořadí: nejprve se napíše test definující očekávané chování, teprve pak implementace, která test splní (Beck, 2000). Tím se test stává specifikací požadavku, nikoli zpětnou kontrolou napsaného kódu, a vývojář je nucen vyjádřit záměr ještě před tím, než ho začne realizovat. Z této role plyne i praxe, že existující testy se v rámci implementace neupravují, jinak ztrácejí svou specifikační funkci. Meta-analýza 27 studií Rafiqueho a Mišiće (Rafique & Mišić, 2013) ukazuje, že TDD má pozitivní efekt na kvalitu kódu měřenou hustotou defektů, přičemž efekt je výraznější v průmyslových projektech než v akademických experimentech.

Code review a traceability pull requestů.

Začleňování změn do hlavního kódu strukturuje code review, dnes ve své *modern code review* podobě integrované do pull requestů (Bacchelli & Bird, 2013). Kontrola probíhá průběžně nad menšími změnami, ne jako samostatná inspekce celé implementace. Aby review plnilo roli auditovatelného mostu mezi zadáním a kódem, je potřeba propojit pull request zpět na issue, ze kterého změna vznikla, typicky odkazem v těle PR (**Closes #N**) (Gotel & Finkelstein, 1994), a doplnit popis, který reviewerovi sděluje co a proč se měnilo. Tato traceability je předpokladem toho, aby bylo možné kdykoli zpětně rekonstruovat, proč byla změna provedena a jak bylo ověřeno, že odpovídá zadání.

2.1.2 Produktová kvalita

Zatímco procesní stránka popsaná v předchozí podsekcí sleduje, jak změna vznikla, produktová stránka se obrací k vlastnostem samotných výstupů vývoje. Produktovou kvalitou se rozumí vlastnosti zdrojového kódu, testů a dokumentace. Tyto vlastnosti nelze redukovat na jediné měřítko, protože různí stakeholderi kladou důraz na jiné aspekty: koncový uživatel a zadavatel sledují především chování za běhu, tedy zda software dělá to, co se od něj očekává a zvládá okrajové případy, zatímco vývojář a maintainer hodnotí čitelnost a modifikovatelnost kódu, se

kterým budou dál pracovat (Kitchenham & Pfleeger, 1996). Tyto pohledy jsou navíc vzájemně nezávislé. Funkčně správný software může být obtížně udržitelný, a naopak elegantně napsaný kód nemusí dělat to, co má.

Standardizovaný rámec pro tuto vícevrstvou povahu poskytuje ISO/IEC 25010 (ISO/IEC, 2023). Model rozkládá kvalitu produktu do osmi charakteristik: funkční vhodnost, výkonnostní efektivita, kompatibilita, použitelnost, spolehlivost, bezpečnost, udržitelnost a přenositelnost. Tyto charakteristiky lze posuzovat odděleně. Pro tuto práci jsou nejrelevantnější dvě. *Funkční vhodnost* vyjadřuje, do jaké míry software poskytuje funkce naplňující stanovené potřeby, a v praxi se zajišťuje testováním. *Udržitelnost* vyjadřuje, s jakou účinností lze software dále upravovat při opravách, zlepšeních nebo přizpůsobování změnám, a zajišťuje se především statickou analýzou. Obě deterministické vrstvy doplňuje lidské peer review pro aspekty, které pravidly nelze vyjádřit. Následující odstavce postupně popisují, jak se každá z těchto vrstev používá.

Funkční vhodnost: testování.

Funkční vhodnost se zajišťuje testováním, tedy systematickou kontrolou chování implementace vůči očekávání. Testovací přístupy se podle toho, na čem staví, dělí na dva základní typy: *black-box* (funkcionální) testování pracuje přes veřejné rozhraní a ověřuje pozorovatelné výstupy nezávisle na vnitřní struktuře implementace, zatímco *white-box* (strukturální) testování vychází ze znalosti kódu a cíleně pokrývá jeho cesty (Ammann & Offutt, 2016).

V rámci *black-box* přístupu se ustavila praxe *behavior-driven testing*, kde testy explicitně formalizují očekávané chování jako specifikaci. Zdrojem této specifikace jsou typicky *acceptance criteria* (AC), tedy popisy chování ze zadání, často ve formátu *Given/When/Then*. Testy odvozené z AC tak slouží zároveň jako spustitelná dokumentace požadavků (Solis & Wang, 2011).

Samotný průchod testů ovšem vypovídá primárně o testované implementaci, nikoli o síle testovací sady. První vrstvou hodnocení testů je strukturální coverage (*line, branch*), která ukazuje, jak velkou část kódu testy vykonaly. Coverage sama o sobě ale neříká, zda by testy skutečnou chybu zachytily: vykonaný řádek bez vhodné aserce projde i tehdy, když implementace dělá něco zcela jiného, než má. Inozemtseva a Holmes na velkém vzorku Java projektů empiricky doložili, že coverage nekoreluje silně s efektivitou testovací sady při detekci chyb (Inozemtseva & Holmes, 2014).

Mutation testing tuto slabinu doplňuje druhým pohledem na sílu testovací sady: do programu systematicky zavádí drobné změny (*mutanty*), které simulují běžné programátorské chyby, a sleduje, kolik z nich testy odhalí (Papadakis et al., 2019). Vysoká míra zachycených mutantů indikuje, že testy skutečně ověřují chování, ne jen pokrývají kód.

Udržovatelnost: statická analýza.

Udržovatelnost se na rozdíl od funkční vhodnosti neprojevuje za běhu, ale ve vlastnostech zdrojového kódu, takže její část lze hodnotit i bez spuštění programu.

Cyklomatická složitost (McCabe, 1976) kvantifikuje strukturální složitost jedné funkce: každá rozhodovací konstrukce (`if`, `while`, `case`, logické operátory `&&` a `||`) přidá k hodnotě jedničku, takže funkce bez větvení má složitost 1. Hodnota zároveň udává minimální počet testovacích případů potřebných k pokrytí všech rozhodovacích cest. V praxi se proto často udržuje práh 10 na funkci (McConnell, 2004); nad ním funkce statisticky vykazují vyšší míru defektů a obtížnější review.

Vedle metrik složitosti se do statické analýzy řadí i linting, který kontroluje porušení jazykových a projektových konvencí, a typová kontrola, která zachycuje nekonzistence v rozhraních a práci s datovými typy ještě před spuštěním programu. Beller et al. (Beller et al., 2016) ukázali na rozsáhlé studii open-source projektů, že tyto nástroje jsou v praxi široce používanou součástí vývojové infrastruktury.

Strukturální nástroje ale pokrývají jen ty aspekty udržovatelnosti, které lze formálně vyjádřit pravidlem. Vlastnosti jako čitelnost, srozumitelné pojmenování, oddělení zodpovědností nebo přiměřenost návrhových rozhodnutí přesahují možnosti čistě deterministických nástrojů (McConnell, 2004), protože vyžadují posouzení záměru, kontextu a srozumitelnosti z pohledu jiného vývojáře.

Co determinismus nezachytí: peer review.

K aspektům, které deterministické nástroje nezachytí, slouží lidská kontrola kódu prováděná peer reviewerem: jiný vývojář než autor čte navrhovanou změnu, typicky v podobě komentářů u pull requestu, a posuzuje ji před začleněním. Tutéž aktivitu zmiňuje sekce 2.1.1 z procesního úhlu, kde je relevantní samotný fakt, že review proběhlo, a jeho navázání na issue. Z produktového hlediska zajímá obsah review: konkrétní připomínky k pojmenování, dekompozici nebo přiměřenosti návrhových rozhodnutí, tedy přesně k těm vlastnostem, ke kterým se statická analýza nedostane. Hodnotu této kontroly kvantifikuje McIntosh et al. (McIntosh et al., 2016): kód, který neprošel review, vykazuje měřitelně vyšší míru defektů po vydání. Jak se tato lidská vrstva hodnocení mění s nástupem AI agentů, popisuje sekce 2.2.3.

2.1.3 Zdrojová dimenze

Zdrojovou dimenzí se rozumí vstupy, které jsou nutné k tomu, aby produkt vznikl: čas, lidské úsilí a použití výpočetních prostředků (Fenton & Bieman, 2014). Tato dimenze je samostatná v tom smyslu, že ani z hotového produktu, ani ze záznamu procesu nelze přímo odvodit, kolik úsilí a času vznik stál.

Tradiční proxy a jejich limity.

V tradičním softwarovém inženýrství se zdroje měří přes kvantitativní proxy. Cost models jako COCOMO (Boehm, 1981) a jeho aktualizace COCOMO II (Boehm et al., 2000) odhadují pracnost na základě velikosti projektu a sady korekčních faktorů, v agilních týmech se používají relativní odhady (*story points*) a ukazatele průchodu prací (*cycle time*, *lead time*). Predikční přesnost těchto modelů je ale dlouhodobě sporná. Systematický přehled 304 studií Jørgensena a Shepperda (Jørgensen & Shepperd, 2007) ukazuje, že odhady pracnosti běžně vykazují vysokou chybu a že strojově spočítané modely často neporazí lidský odhad zkušeného inženýra. Společným omezením všech těchto měřítek je navíc to, že redukují vícerozměrný jev (čas, úsilí, kvalita výstupu, dopad na tým) na jedinou osu, a samy o sobě nezachytí, zda je vývoj efektivní v širším slova smyslu.

Víceosé měření a přechod ke zdrojům agentů.

Reakcí na tato omezení jsou rámce DORA (Forsgren et al., 2018) a navazující SPACE (Forsgren et al., 2021), které explicitně varují před redukcí výkonnosti na jedinou metriku a místo toho pracují s několika osami současně. Stejný princip víceosého měření přenášíme i na hodnocení agenta, ale s posunem v obsahu zdrojové dimenze: lidské úsilí v klasickém Fentonově smyslu u autonomního agenta odpadá a jeho místo zaujímají tokeny jako proxy výpočetního úsilí, wall-clock čas běhu a stabilita session, k níž v agentním prostředí přibývá kontextové okno jako vzácný zdroj (sekce 3.2.3). Tyto osy a jejich specifika podrobněji popisuje sekce 2.2.2.

2.2 AI coding agenti

Předchozí sekce popsala kvalitu software v tradičním vývoji. AI coding agenti tento kontext mění: kód nepíše člověk, ale autonomní systém řízený instrukcemi. Tři dimenze měření z předchozí sekce zůstávají platné, mění se však jejich projev. Produktové vzorce kódu, procesní stopa v repozitáři i proxy nákladů vznikají u agenta jinak než u lidského vývojáře.

2.2.1 Základní pojmy

Velký jazykový model (*large language model*, LLM) je neuronová síť trénovaná na rozsáhlých textových korpusech (Vaswani et al., 2017). Na základě předchozího kontextu predikuje pravděpodobný následující token, a tím generuje koherentní text, včetně zdrojového kódu. Samotný LLM je pasivní: odpovídá na dotazy, ale nemůže samostatně číst soubory, spouštět příkazy ani měnit stav projektu. Predikce navíc není deterministická, takže ani explicitní instrukce nezaručuje, že ji model dodrží.

LLM-based agent model rozšiřuje o schopnost interagovat s prostředím (Liu et al., 2024): plánuje kroky, volá nástroje prostřednictvím tzv. *tool calls* (Schick et al., 2023) a na základě pozorovaného výsledku rozhoduje o dalším kroku v cyklu Thought → Action → Observation (Yao et al., 2023). Vrstvu, která *tool calls* zprostředkovává a spravuje stav session, tvoří *harness*, tedy běhové prostředí poskytované nástrojem. Harness agentovi zpravidla nabízí minimálně tři typy nástrojů: čtení souboru, zápis souboru a spuštění shell příkazu. Přes ně může v repozitáři spouštět testy, používat verzovací systém i volat externí API. Tyto akce po sobě zanechávají měřitelné stopy (commity, větve, pull requesty) a jsou současně zdrojem typických selhání: nesprávně zvolená větev, neoprávněná modifikace existujícího testu nebo vynechaný krok pracovního postupu.

Harness výrazně ovlivňuje výkon agenta. Yang et al. (Yang et al., 2024) u SWE-agent ukazují, že i drobné úpravy rozhraní mezi agentem a prostředím (*agent-computer interface*) posouvají úspěšnost na reálných úlohách o jednotky procentních bodů. Harness také zaznamenává průběh session jako strukturovaný transcript, který slouží jako audit log běhu agenta. Spolu s obsahem kontextového okna tvoří dvě odlišné vrstvy řízení chování: harness skrze design rozhraní a *tool calls*, instrukce skrze textový obsah, který agent dostane jako vstup.

Předmětem této práce jsou *autonomní coding agenti*, tedy systémy, které na základě specifikace a instrukcí samostatně implementují softwarový projekt (např. Claude Code, OpenHands, SWE-agent) (Guo et al., 2025). Na rozdíl od asistenčních nástrojů (GitHub Copilot) agent provádí celý vývojový proces, nikoli jen jednotlivé kroky. Jeho výstupy lze proto hodnotit ve všech třech dimenzích zavedených v sekci 2.1: produktové (jaký kód vznikl), procesní (jak při tom postupoval) a zdrojové (kolik tokenů a kontextu spotřeboval). Posun každé z nich popisuje následující sekce.

2.2.2 Jak agenti mění softwarové inženýrství

Přechod od lidského vývojáře k autonomnímu agentovi posouvá softwarové inženýrství ve všech třech dimenzích zavedených v sekci 2.1. Jin et al. (Jin et al., 2024) v přehledu 124 studií ukazují, že agenti pronikají do šesti oblastí SE: požadavků, generování kódu, autonomního rozhodování, návrhu, generování testů a údržby. Tato práce se zaměřuje na fáze generování kódu a testů, ke kterým je dnes k dispozici nejvíce empirických studií agentního chování. Tento posun zároveň přináší typická selhání, která se v každé dimenzi projevují jinak. Jejich společným pozadím jsou omezení, která agent dědí z LLM: omezené kontextové okno, absence implicitní znalosti projektu a pravděpodobnostní povaha výstupu.

Procesní stránka se mění v tom, kdo provádí jednotlivé kroky, a v tom, jakou stopu po sobě zanechávají. Typická procesní selhání dokládají empirická pozorování: agent-generované pull requesty se od těch lidských liší mírou nadbytečnosti, konzistencí commit zpráv a ochotou reviewera je přijmout (Ehsani et al., 2026; Watanabe et al., 2025).

Produktová stránka zůstává vymezena stejnými charakteristikami (viz sekce 2.1.2), ale typické vzorce agent-generovaného kódu jsou jiné než u lidského vývojáře. Li et al. (B. Li et al.,

2026) na 1210 merged agent-PR z reálných repozitářů zjistili, že i přijaté patche nesou vyšší výskyt code smells, duplikací a strukturálních problémů, které merge samotný neodhalí. Tato produktová selhání se proto v literatuře hodnotí kombinací automatizovaných benchmarků (sekce 2.2.3) s post-hoc statickou analýzou a s hodnocením od LLM-judgů či lidských reviewerů.

Zdrojová stránka dostává u agentů nové proxy. Na místě člověko-hodin stojí spotřebované tokeny a čas běhu na úlohu, kontextové okno a rate limit vystupují jako vzácné zdroje, které ovlivňují, kolik úloh a jak dlouhých agent zvládne. Zdrojová selhání pak souvisejí s neefektivním řízením kontextu: Lindenbauer et al. (Lindenbauer et al., 2025) pozorují, že jednoduché skrývání starších výstupů z tool calls (*observation masking*) dosáhne srovnatelné úspěšnosti jako jejich shrnutí pomocí LLM za polovinu nákladů.

Jak se schopnosti agentů konkrétně měří dnes a jaké mezery v tomto měření zůstávají, shrnuje následující sekce.

2.2.3 Hodnocení schopností agentů

Hodnocení autonomních coding agentů prošlo několika vlnami. Foundational benchmark SWE-bench (Jimenez et al., 2024) testuje schopnost agenta vytvořit patch na 2 294 úlohách z reálných GitHub repozitářů, kde agent dostane popis problému (GitHub issue) a má vytvořit patch, který projde testy. Aktuálně se jako standard prosazuje Terminal-bench (Merrill et al., 2026), 89 tvrdých úloh v reálných terminálových prostředích, kde i frontier modely zatím překračují 50 % úspěšnosti jen výjimečně. Starší HumanEval (Chen et al., 2021) zůstává referencí pro generování funkčně správného kódu na izolovaných programovacích úlohách. Společné je všem binární hodnocení: patch projde nebo neprojde.

Tyto benchmarky měří *funkční korektnost*: zda výsledek dělá to, co má. Neměří však, jak agent pracoval (dodržoval proces?), jakou strukturální kvalitu má výsledný kód (je udržitelný?) ani za jakou cenu výsledku dosáhl (kolik tokenů spotřeboval?). Tři empirické studie zmíněné v kapitole 1 (METR (METR, 2026), Li et al. (B. Li et al., 2026), Ehsani et al. (Ehsani et al., 2026)) shodně dokládají, že pass/fail výsledek nezachycuje, zda by patch obstál v code review, byl udržitelný nebo vznikl auditovatelným postupem.

Existují benchmarky, které hodnocení rozšiřují nad rámec pass/fail. SWE-bench Pro (Scale AI, 2025) přidává strukturované požadavky, ACE-Bench („ACE-Bench: End-to-End Feature Development Benchmark“, 2025) testuje end-to-end vývoj a FeatureBench („FeatureBench: Benchmarking Agentic Coding for Complex Feature Development“, 2026) hodnotí implementaci celých funkcí. Yin et al. (Z. Yin et al., 2025) jdou ještě dál a hodnotí sedm agentních frameworků na třech osách (úspěšnost, efektivita reasoning kroků a tokenová režie); jejich pohled na proces je ale omezen na trasu reasoning kroků agenta a strukturální kvalita kódu v hodnocení chybí. Žádný z uvedených benchmarků tedy systematicky neměří kvalitu procesu (jak agent organizoval práci), kvalitu produktu nad rámec funkční korektnosti (udržovatelnost, čistota) a spotřebu zdrojů současně. Tato mezera naznačuje potřebu metrického rámce, který by dimenze procesu, kvality produktu a zdrojů pokrýval současně. Návrh takového rámce

a jeho operacionalizaci v kontextu této práce popisuje kapitola 3.

LLM-as-judge.

Některé dimenze kvality, jako srozumitelnost commit zpráv, kvalita dokumentace nebo vhodnost návrhových vzorů, nelze spolehlivě hodnotit deterministickými nástroji. Jedním z přístupů je využití LLM jako hodnotitele (*LLM-as-judge*). Zheng et al. (Zheng et al., 2023) ukázali, že LLM dokáže hodnotit kvalitu výstupů na úrovni srovnatelné s lidskými hodnotiteli, ale identifikovali tři systematické biasy: *position bias* (preferuje odpověď na určité pozici), *verbosity bias* (preferuje delší odpovědi) a *self-enhancement bias* (preferuje vlastní výstupy). Panickssery et al. (Panickssery et al., 2024) prokázali korelaci mezi schopností modelu rozpoznat vlastní výstupy a mírou, do jaké je preferuje, což naznačuje kauzální vztah mezi self-recognition a self-preference. Verga et al. (Verga et al., 2024) navrhli mitigaci formou panelu diverzních modelů (PoLL), který dosahuje lepší shody s lidským hodnocením než jednotlivý model. Praktickou implikací je volba hodnotícího modelu z odlišné modelové rodiny, než ve které pracuje hodnocený agent.

Test oracle problem.

Test oracle problem je klasická obtíž testování: kdo nebo co určuje správný výsledek, proti kterému se test porovnává? U autonomního agenta se zostruje tím, že agent generuje implementaci i testy současně. Pokud testy odvodí z toho, jak se kód právě chová, jejich průchod potvrzuje jen vzájemnou konzistenci, ne shodu se specifikací. Mathews et al. (Mathews & Nagappan, 2024) dokládají, že generátory testů zahazují právě ty testy, které by chybu odhalily. Až 68,1 % výsledných testovacích sad pak validuje chybné chování místo jeho odhalení. Obranou je odvozovat očekávané hodnoty ze specifikace (*spec-first TDD*, sekce 2.1.1). Specifikace tím vystupuje jako externí oracle, který agent nemůže přepsat.

Benchmarky i doplňkové mechanismy z této sekce hodnotí převážně izolované úlohy a jednotlivé aspekty výstupu. Pro vyhodnocení agenta v kontextu jedné case study je třeba metrik pokrývajících dimenze produktu, procesu i zdrojů současně. Návrh takové sady je obsahem kapitoly 3.

2.2.4 Instrukce jako nezávislá proměnná

Disciplína zabývající se formulací textových vstupů pro velký jazykový model se v literatuře označuje jako *prompt engineering* a její rozšíření o správu celého kontextového okna jako *context engineering*. Tato práce používá nadřazený pojem *instrukce*, který obě tyto disciplíny zastřešuje.

Autonomní agent nemá *tacit knowledge*, tedy implicitní znalosti, které lidský vývojář získává

zkušeností: jaké konvence tým dodržuje, proč je kód strukturovaný určitým způsobem nebo jaké chyby se v projektu opakují. Nonaka a Takeuchi (Nonaka & Takeuchi, 1995) popisují převod tacit knowledge na explicitní jako klíčový proces v organizacích. Pro agenty je tento převod nutný v plném rozsahu: veškerý kontext, který agent potřebuje, musí být zapsán explicitně. Hassan et al. (Hassan et al., 2025) tuto dualitu formalizují v rámci frameworku SASE, který rozlišuje dva režimy vývoje. V *SE4H (Software Engineering for Humans)* je vývojářem člověk a kontext čerpá z meetingů, code review a zkušenosti. V *SE4A (Software Engineering for Agents)* je vývojářem autonomní systém, který pracuje výhradně s tím, co dostane jako vstup.

Scaffolding a soubory s instrukcemi.

Scaffolding označuje strukturované artefakty v prostředí agenta, které jeho chování řídí. V praxi jde nejčastěji o soubory s instrukcemi umístěné v repozitáři (např. **AGENTS.md**, **CLAUDE.md**), které se v poslední době prosazují jako de facto standard napříč většinou coding agentů. Tyto soubory definují roli agenta, pracovní postup, konvence a omezení. Na rozdíl od jednorázových promptů jsou tyto soubory verzované a iterativně upravované, což z nich činí „living documents“, jejichž vývoj lze zpětně sledovat.

Komponenty instrukcí.

Mao et al. (Mao et al., 2025) analyzovali 2 163 produkčních prompt šablon a identifikovali sedm typů komponent: Role, Directive, Context, Workflow, Output, Constraints a Examples. Zjistili, že Role a Directive se nejčastěji objevují na začátku dokumentu a že Workflow (procedurální kroky) je nejúčinnější komponentou pro složité úkoly. Toto zjištění potvrzují Li et al. (X. Li et al., 2026), kteří ukazují, že procedurální instrukce jsou efektivnější než popisná dokumentace.

Empirie účinnosti.

Empirická evidence o účinnosti instrukčních souborů je smíšená a závisí na tom, co se měří a jaký je obsah souboru. V efektivitě běhu Lulla et al. (Lulla et al., 2026) pozorují, že přítomnost **AGENTS.md** zkracuje mediánovou dobu, kterou agent potřebuje na splnění úlohy, o 28,6 %. V úspěšnosti řešení úlohy ale rozhoduje původ obsahu: LLM-generované instrukce úspěšnost snižují a vývojářem psané přinášejí jen marginální zlepšení (Gloaguen et al., 2026), kdežto kurátorovaný obsah v benchmarku SkillsBench zvyšuje úspěšnost o 16,2 procentních bodů a automaticky generovaný nemá žádný efekt (X. Li et al., 2026). Rozhodující je tedy obsah, struktura a relevance, ne pouhá přítomnost souboru. Účinnost jednotlivých typů obsahu přitom dosud nebyla izolována.

Specificita a citlivost.

Konkrétnost instrukce je sama o sobě faktor: Kim et al. (Kim, 2025) v benchmarku DETAIL a Zi et al. (Zi et al., 2025) na úlohách s kódem shodně ukazují, že vyšší *specificita* zvyšuje úspěšnost agenta, ale přílišný detail může omezit jeho reasoning. Instrukce se tak pohybují po ose od volné nápovědy po explicitní příkaz, přičemž optimální bod závisí na úloze i modelu.

Modely jsou kromě toho citlivé na samotnou formulaci. Razavi a Fard (Razavi & Fard, 2025) dokládají, že drobné změny promptu mohou dramaticky změnit chování modelu. Breunig et al. (Breunig, 2025) na to navazují praktickým doporučením: když agent pravidlo ignoruje, nepomůže ho zopakovat, je třeba ho přeformulovat tak, aby se stalo součástí pracovního postupu.

Mechanismy účinku.

Výzkum promptingu navíc naznačuje, že instrukce nepůsobí na chování modelu jedním mechanismem. Wei et al. (Wei et al., 2022) ukazují, že i krátká nápověda (“think step by step”) aktivuje reasoning, který se bez ní neprojeví, a Min et al. (Min et al., 2022) dokládají, že demonstrace v in-context learning působí spíš jako strukturální vodítka než jako přímé příkazy. Instrukce tedy nemusí působit jen jako explicitní příkazy: mohou také strukturovat řešení nebo připomínat schopnosti, které se bez vhodného podnětu neprojeví. Která forma instrukce bude v konkrétním případě účinnější, nelze rozhodnout předem.

Otevřenou otázkou proto není jen to, zda instrukce obecně pomáhají, ale jak jejich obsah systematicky navrhovat a jak poznat, které složky skutečně mění chování agenta.

Z této kapitoly plynou dvě motivace pro zbytek práce: současné hodnocení agentů nepokrývá tři dimenze měření společně a návrh instrukcí, ač hlavní páka chování, nemá ustanovený systematický postup. Oba problémy adresuje kapitola 3, která zavádí sadu metrik pokrývajících tři dimenze a iterativní postup návrhu instrukcí řízený těmito metrikami.

3. Metodika

Předchozí kapitoly ukázaly potřebu vícerozměrného hodnocení agenta a roli instrukcí jako nezávislé proměnné. Tato kapitola na nich staví metodiku.

Sekce 3.1 zdůvodňuje volbu případové studie jako výzkumné strategie, představuje zvolený projekt, fáze studie (pilot a ablance) a strukturu iterativního cyklu. Sekce 3.2 definuje sadu 19 metrik ve třech kategoriích (proces, produkt, zdroje). Sekce 3.3 popisuje operační detail: fixní proměnné, záznam běhu a diagnostickou proceduru. Sekce 3.4 shrnuje metriky tabulkou a sekce 3.5 diskutuje hrozby validity.

3.1 Výzkumný přístup

Tato sekce postupně odpovídá na tři otázky: jakou výzkumnou strategii volíme a proč (sekce 3.1.1), na jakém případě strategii ověřujeme (sekce 3.1.2) a jak vypadá iterativní cyklus, který v rámci případu opakujeme (sekce 3.1.3).

3.1.1 Volba výzkumné strategie

Z tří cílů formulovaných v kapitole 1 plynou společné nároky na metodiku. Cíl 1, návrh sady metrik pokrývající proces, kvalitu produktu a zdroje, lze ověřit jen tehdy, když metriky reagují na změny v chování; to vyžaduje porovnání více běhů s různými variantami instrukcí. Cíl 2, iterativní postup návrhu instrukcí řízený těmito metrikami, vyžaduje navíc, aby běhy tvořily sekvenci, v níž lze sledovat vztah měření → diagnóza → úprava → další měření. Cíl 3, ablance složek instrukcí, stojí na schopnosti porovnat běhy lišící se v jediné složce. Všechny tři cíle tedy společně kladou na metodiku dva požadavky: opakované spouštění agenta s různými variantami instrukcí na témže projektu a podrobnou analýzu každého běhu napříč třemi dimenzemi (proces, produkt, zdroje).

Tomuto profilu odpovídá případová studie. Řízený experiment s počtem běhů potřebným pro statistickou izolaci by byl vzhledem k ceně jedné iterace (tisíce tokenů, desítky minut) nepraktický. Volíme proto hloubku před šířkou: jeden projekt s detailní analýzou každého běhu (git historie, transcript, naměřené metriky). Yin (R. K. Yin, 2018) pro tento typ výzkumu používá pojem *analytická generalizace*: na jednom případě ukazujeme princip, který lze dále ověřovat na dalších případech. Praktický přínos této práce proto formulujeme jako demonstraci proveditelnosti, zatímco analytická generalizace vymezuje způsob, jak opatrně číst přenositelné principy. Všechny běhy vycházejí ze stejné specifikace projektu, liší se verzí instrukcí, výslednou git historií a naměřenými hodnotami metrik.

3.1.2 Příklad a jeho prostředí

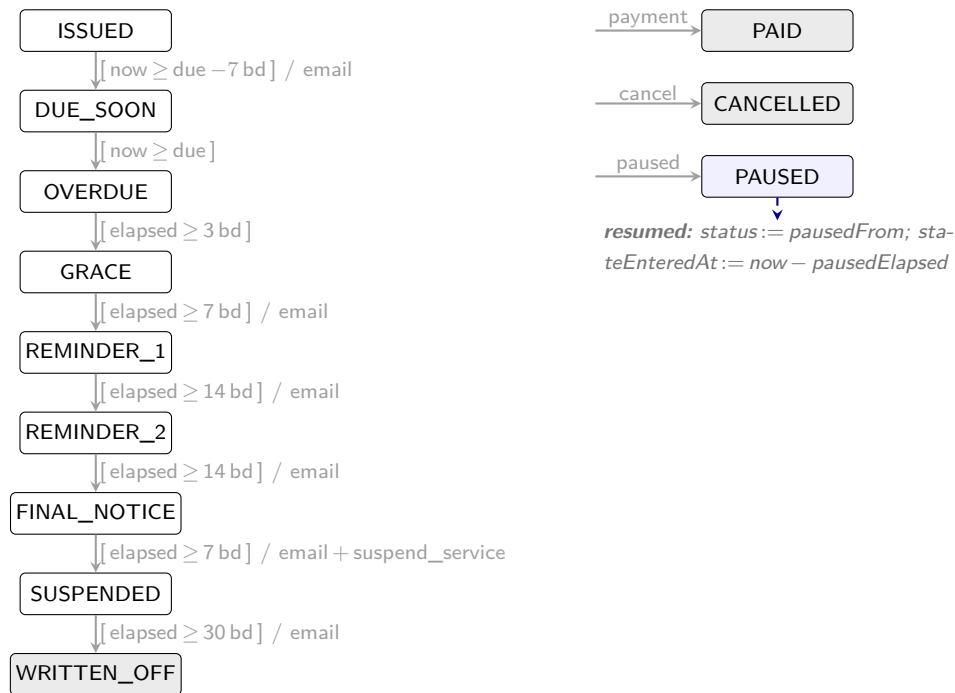
Případová studie potřebuje projekt, na kterém lze opakovaně spouštět agenta s různými instrukcemi a objektivně měřit výsledky. Projekt musí mít deterministickou logiku (stejný vstup vždy dá stejný výstup), aby bylo možné ověřit korektnost automatizovanými testy a mutation testingem. Zároveň musí být dostatečně malý pro více experimentálních běhů, ale obsahovat reálné nuance (hraniční případy, doménová pravidla), aby měl agent co řešit.

Zvolili jsme systém upomínek faktur: systém pro automatické odesílání připomínek k nezaplaceným fakturám. Obsahuje stavový automat (nová, po splatnosti, upomínaná, eskalovaná), časové výpočty (pracovní dny, ochranné lhůty) a pravidla pro eskalaci. Současně jde o úlohu dostatečně bohatou na netriviální rozhodování, ale stále zvládnutelnou pro opakované běhy: agent musí kombinovat stavové přechody, časové výpočty a testování hraničních případů, takže metriky zachycují víc než průchod jednoduchou CRUD implementací.

Specifikace projektu tvoří fixní zadání, které agent dostává v Issue #1. Issue má dvě části: **Requirements** (co business potřebuje) a **API Contract** (co musí implementace exportovat). API kontrakt je současně jediným technickým omezením implementace a vstupem pro metriku Q1 (API contract match).

Doménová logika.

Systém eskaluje neuhrazené faktury sérií stavů od přátelské upomínky před splatností přes stupňující se urgentnost až po pozastavení služby a odpis pohledávky. Každá faktura má vlastní nezávislou instanci, časové prodlevy se počítají v pracovních dnech (víkendy a konfigurovatelné svátky se přeskakují automaticky v aritmetice timeoutů, nejsou samostatným stavem). Hlavní eskalační větev má 9 stavů spojených 8 časovými přechody (obrázek 3.1). Popisky šipek používají notaci **[guard] / effect**: guard v hranatých závorkách udává timeout v pracovních dnech (*bd* = business days) od vstupu do předchozího stavu, effect za lomítkem je akce vrácená volajícímu (typicky odeslání e-mailu, u přechodu do **SUSPENDED** navíc pozastavení služby). Eskalační přechody spouští událost **tick** při splnění guardu; alternativně **manual_advance** posune stav na další bezpodmínečně (bypass timeoutu, v diagramu nezakreslen). Vedle hlavní větve jsou ze všech aktivních stavů kdykoliv dostupné **PAID** a **CANCELLED** (z 9 stavů, vše kromě terminálů) a **PAUSED** (jen ze 6 aktivních: **OVERDUE** až **SUSPENDED**).



Obrázek 3.1: Stavový automat systému upomínek faktur: hlavní eskalace shora dolů (vlevo) a tři boční stavy dostupné z většiny aktivních (vpravo). Notaci a zdrojové stavy popisuje text výše.

API kontrakt.

Aby bylo možné agentovu implementaci porovnávat s referenční sadou behavioral testů (Q2), specifikace fixuje veřejné rozhraní: systém je čistá funkce `process(state, event, now)`, která pro daný stav, událost a aktuální čas vrátí nový stav a seznam akcí. Pevné jsou názvy 12 stavů (9 v hlavní eskalační větvi a 3 boční: PAID, CANCELLED, PAUSED), 6 typů událostí (`tick`, `payment_received`, `invoice_cancelled`, `dunning_paused`, `dunning_resumed`, `manual_advance`) a 3 typy akčních deskriptorů (`send_email`, `suspend_service`, `resume_service`). Stav nese kromě `status` pole pro datum splatnosti, čas vstupu do aktuálního stavu, konfiguraci a při pauze `pausedFrom` s `pausedElapsed`. Vnitřní implementace (modulární rozdělení, funkční vs. objektový styl) zůstává na agentovi: kontrakt je úzký právě v tom, co testy potřebují volat a pozorovat, a všechno ostatní nechává otevřené.

Implicitní pravidla domény.

Stavový automat je sám o sobě lineární a implementačně přímočarý. Obtížnost úlohy leží jinde: ve dvou implicitních pravidlech, která specifikace klade na chování implementace a jejichž přehlédnutí vede ke kódu, který sice prochází triviálními případy, ale na hraničních selhává. Za prvé, pause/resume musí přes pole `pausedElapsed` zachovat uplynulý čas: bez něj by se musel řešit trikem (posun `stateEnteredAt` zpět v čase), který funguje pro jednu pauzu, ale selhává při opakovaném pause/resume, protože druhá pauza ztratí historii první.

Za druhé, všechny timeouty se počítají v pracovních dnech, takže +14 bd od pátku není čtrnáct kalendářních dnů a hranice mezi víkendem a ponděním jsou off-by-one zóna. Obě pravidla jsou plně určená API kontraktem; jejich zachycení ale vyžaduje doslovné čtení typů a doménového slovníku, ne odvozování z diagramu automatu.

Acceptance criteria.

Specifikace obsahuje 25 acceptance criteria ve formátu Given/When/Then. Pokrývají sedm okruhů: časové přechody eskalace, platby, terminální stavy, storno, pauza/obnovení, manuální postup a výpočet pracovních dní. Ukázka jednoho kritéria z okruhu pauza/obnovení:

```
Given an invoice is in PAUSED state
When a dunning_resumed event occurs
Then the state transitions back to the state it was in before pausing
And the timeout resumes from where it left off
```

Formát Given/When/Then usnadňuje převod požadavků do referenčních testů (Q2 (referenční test pass rate)) i mapování acceptance criteria na agentovy vlastní testy (Q4 (AC coverage)). Doménový slovník (9 pojmů: dunning, grace period, business days, action descriptor aj.) sjednocuje terminologii mezi specifikací a implementací.

Out of scope.

Specifikace explicitně vylučuje šest oblastí: opakování plateb, úroky a poplatky z prodlení, částečné platby, odesílání e-mailů, plánování (cron) a persistenci dat. Sekce out of scope omezuje riziko, že agent rozšíří řešení nad rámec sledovaného zadání, a drží úlohu na úrovni čisté business logiky.

Referenční testy jako oracle.

Pro metriku Q2 (referenční test pass rate) (sec 3.2.2) byla souběžně se specifikací připravena sada 42 referenčních testů metodou spec-first TDD (Mathews & Nagappan, 2024): očekávané hodnoty pochází ze specifikace, ne z pozorování kódu. Testy jsou agentovi neviditelné a spouští se až po běhu samostatným skriptem, takže slouží jako externí oracle, který agent nemůže přepsat (sec 2.2.3). Mutation testing nad referenční test suite dosáhl 91,9 %, což indikuje, že referenční testy samy detekují cílené poruchy v doménové logice. Tato hodnota slouží jako pojistka kvality měřicího nástroje, ne jako srovnávací baseline pro agentovy běhy.

V tomto designu vystupují tři roviny, které se snadno zaměňují. *Instrukce* v souboru `AGENTS.md` jsou nezávislou proměnnou: měníme je mezi běhy a sledujeme dopad. *Chování agenta* (jak

implementuje, jak strukturuje práci a jaký kód vytváří) je závislou proměnnou, kterou měříme. Systém upomínek faktur slouží jako testovací prostředí, na kterém chování pozorujeme; není předmětem zkoumání.

3.1.3 Fáze a iterativní cyklus

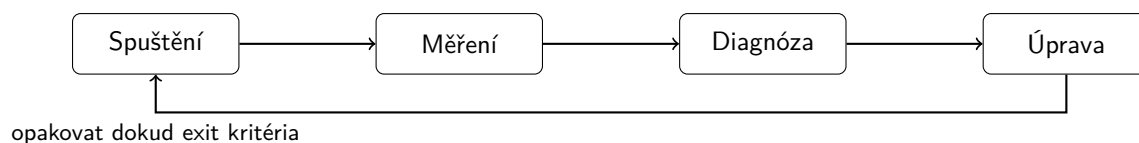
Případová studie postupuje ve dvou fázích. *Pilotní fáze* ověřuje proveditelnost cílů 1 a 2: opakovaně spouští agenta a upravuje instrukce, dokud agent na zvolených kritériích nedosáhne stabilního zlepšení. *Ablace* pak ověřuje cíl 3: na fungující verzi instrukcí systematicky odebírá jednotlivé složky a měří dopad. Bez pilotu by ablace neměla z čeho vycházet, bez ablace bychom o redundanci složek mohli jen spekulovat.

Pilotní fáze.

Pilotní fáze postupuje proti exit kritériím ve sloupci *Exit kritérium* tabulky 3.10. Tato kritéria představují principiální ideál: striktní 42/42 referenčních testů, nulové lint warnings, nulové porušení složitosti. Sám zvolený projekt nemusí být v tomto smyslu plně dosažitelný v jediném autonomním běhu. Pro praktické řízení iterací proto pracujeme s *operačními prahy* ($Q2 \geq 37/42$, $Q3 \geq 70\%$, $Q4 \geq 24/25$, $Q5 \leq 1$, $Q7 \leq 1$ porušení), které lépe rozlišují běhy mezi sebou a odpovídají barevnému kódování souhrnné tabulky a vizualizací v kapitole 5. Pilotní fázi proto vyhodnocujeme dvěma pohledy současně: která *striktní* kritéria běh splnil (kapitola 5, per-běh tabulky pass/fail kritérií) a zda splnil operační prahy (graf 5.2). Pilotní fáze končí, když některý běh splní celý procesní checklist P1–P5 a vykáže operačně rozeznatelné zlepšení proti baseline; další iterace by přinášely jen drobné lokální úpravy. Tato definice odpovídá proveditelnosti na jednom případě, neimplikuje obecnou platnost prahů pro jiné agenty a projekty.

Ablace.

Z fungující sady instrukcí (výstup pilotní fáze) systematicky odebíráme jednotlivé složky a měříme dopad na chování agenta: potřebuje agent danou část instrukcí, nebo je redundantní? Konkrétní variace nelze specifikovat předem; vycházejí z diagnostiky pilotních běhů. Pokud se v pilotu ukáže, že agent konzistentně nedodržuje určité pravidlo, ablace testuje, zda jeho odebrání změny i ostatní metriky. Každá ablace se provádí ve dvou nezávislých bězích se stejným nastavením, aby bylo možné odlišit systematický efekt změny od přirozené variability neterministického modelu. I se dvěma běhy nelze dosáhnout statistické průkaznosti (sekce 3.5); výsledky proto interpretujeme jako indikativní, ne kauzální.



Obrázek 3.2: Iterativní cyklus: čtyři kroky jedné iterace a smyčka opakování až do splnění exit kritérií.

Iterativní cyklus.

V obou fázích je vlastní postup iterativní cyklus, protože jediná iterace nestačí ověřit cíl 2. Pokud má sada metrik fungovat jako diagnostický nástroj, musí naměřená hodnota vést k identifikovatelné úpravě instrukcí a další běh musí ukázat, zda úprava zabrala. Iterace má čtyři kroky: Spuštění (běh agenta), Měření (extrakce metrik), Diagnóza (analýza odchylek) a Úprava (změna instrukcí). Cyklus iteraci opakuje, dokud nejsou splněna exit kritéria (obr. 3.2). Substantivní část iterace popisuje sekce 3.3.3.

3.2 Sada metrik

Každý experimentální běh probíhá tak, že agent dostane prázdné GitHub repo, specifikaci v Issue #1 a instrukce v `AGENTS.md`, a autonomně implementuje zadaný projekt. Výstup jednoho běhu (kód, testy, git historie) měříme sadou 19 metrik. Postup běhu a experimentální design popisuje sekce 3.3.

Sada metrik pokrývá tři otázky: jak agent pracoval, co vyrobil a za jakou cenu. Tyto tři otázky odpovídají taxonomii proces / produkt / zdroje popsané v sekci 2.1. Naše sada obsahuje 19 metrik ve třech kategoriích:

- P – proces (P1–P8)** Dodržel agent předepsaný postup? Měříme dodržování pravidel pracovního postupu z instrukcí (P1–P5) a kvalitu procesních výstupů jako commit messages, issue a PR descriptions (P6–P8).
- Q – kvalita produktu (Q1–Q8)** Je kód kvalitní? Měříme funkční korektnost (Q1–Q2), kvalitu testů (Q3–Q4) a udržitelnost kódu (Q5–Q8).
- E – zdroje (E1–E3)** Za jakou cenu? Zaznamenáváme spotřebované tokeny (E1), čas (E2) a počet kompakcí kontextu (E3).

Teoretické základy jednotlivých metrik (proč jsou validní, jaká je evidence) popisuje sekce 2.1. Tato sekce definuje postup měření: jaký nástroj používáme, jaká data jsou vstupem a co přesně je výsledkem. Interpretace přínosu jednotlivých metrik patří až do kapitoly 5.

Tři osy klasifikace metriky.

Každou metriku popisují tři nezávislé osy, které je užitečné držet oddělené. *Způsob měření* odlišuje deterministické metriky (skript, regex, parsing logu, statická analýza) od judge-based metriky hodnocených LLM-as-judge; jde o vlastnost nástroje, ne o roli metriky v hodnocení. *Role v hodnocení* pak rozlišuje pass/fail kritérium s exit prahem od deskriptivního indikátoru, který hodnotu jen zaznamenává bez prahu. *Kategorie* je taxonomie P/Q/E (sekce 2.1). V této práci se pass/fail soud o úspěšnosti běhu opírá o kritéria s exit prahem v kategoriích P a Q (binární i judge-based); celá zdrojová skupina (E1–E3) slouží jako deskriptivní kontext, žádná z jejích metrik nemá exit práh.

3.2.1 Procesní metriky (P1–P8)

Procesní metriky měří *jak* agent pracuje. Instrukce v AGENTS.md definují strukturovaný vývojový postup: agent má odvozovat práci ze specifikace, organizovat ji přes issues a branches, psát testy před implementací a dokumentovat rozhodnutí v commit messages a PR descriptions.

P1–P5: compliance (binární)

Tabulka 3.1 shrnuje deterministické procesní metriky. Všechny mají binární výstup pass/fail.

Tabulka 3.1: Procesní compliance metriky P1–P5

Kód	Co měří	Jak se měří
P1	Issues vznikla před kódem	Porovnání <code>created_at</code> nejstaršího agentova issue (GitHub API) s časem prvního commitu obsahujícího soubory v <code>src/</code> nebo <code>tests/</code> .
P2	Branch per issue	Počet remote branches (bez <code>main</code>) vůči počtu issues; podmínka $branches \geq issues$ (žádné slučování více issues do jedné větve).
P3	Test-first	Primární indikátor: existuje commit s prefixem <code>test:</code> a zároveň <code>feat:</code> . Behavioral trace <code>tddOrderViolations</code> : počet větví, kde první zápis do <code>src/</code> předchází prvnímu zápisu do <code>tests/</code> .
P4	PR linkován na issue	Tělo každého PR (GitHub API) obsahuje regex <code>Closes #N</code> .
P5	Žádné modifikace existujících testů	<code>git diff --diff-filter=M</code> na souborech v <code>tests/</code> ; nově přidané testy se nezapočítávají.

P6–P8: kvalita procesních artefaktů (LLM-as-judge)

Tabulka 3.2 shrnuje judge-based procesní metriky. Postup hodnocení je společný a popsáný v sekci 3.2.4.

Tabulka 3.2: Judge-based procesní metriky P6–P8

Kód	Co hodnotí	Škála
P6	Popisnost commit messages	1–3 (atomicita, konvenční prefix, srozumitelnost co a proč bylo změněno).
P7	Popisnost issue descriptions	1–3 (jasnost scope, přítomnost acceptance criteria, dostatečnost pro implementaci).
P8	Popisnost PR descriptions	1–3 (odkaz na issue, popis co a proč, dostatečnost pro code review).

3.2.2 Produktové metriky (Q1–Q8)

Produktové metriky měří *co* agent vyrobil. Z osmi charakteristik ISO/IEC 25010 (sekce 2.1.2) v této sadě sledujeme dvě: *funkční vhodnost* (Q1–Q4: agent vytváří správné řešení) a *udržovatelnost* (Q5–Q8: agent vytváří kód vhodný k dalšímu pochopení a rozvoji). Ostatní charakteristiky (bezpečnost, výkonnostní efektivita, kompatibilita, použitelnost, spolehlivost, přenositelnost) by se na rozsahu jedné malé případové studie systému upomínek smysluplně měřit nedaly. Sada Q metrik proto pokrývá tři oblasti: funguje implementace správně (Q1–Q2), detekují agentovy testy skutečné chyby (Q3–Q4) a je kód udržitelný (Q5–Q8).

Funkční korektnost (Q1–Q2)

Tabulka 3.3 shrnuje metriky funkční korektnosti.

Tabulka 3.3: Metriky funkční korektnosti Q1–Q2

Kód	Co měří	Jak se měří
Q1	Shoda s API kontraktem	Referenční typy se importují a zkompilují proti agentovu kódu (tsc). Výsledek je binární: typy sedí, nebo ne.
Q2	Pass rate referenčních testů	Vitest spustí 42 testů proti agentovu kódu. Výsledek je počet passing testů z 42. Testy ověřují chování přes veřejné API (black-box, sekce 2.1.2). Konstrukci referenční test suite popisuje sekce 3.1.2.

Poznámka: Q1 je vstupní podmínkou pro Q2: pokud agentův kód neimplementuje správné API, referenční testy nelze ani zkompilovat a výsledek Q2 by byl nesmyslný.

Případová studie rozlišuje dva druhy testů. *Referenční testy* (měřené metrikou Q2) jsou odvozené ze specifikace metodou TDD (Mathews & Nagappan, 2024) a spouštěné po běhu samostatným skriptem, takže pro agenta jsou neviditelné. *Agentovy vlastní testy* píše agent během běhu, jsou viditelné a spustitelné agentem; vztahuje se k nim metrika P5. Referenční testy slouží jako externí oracle, který agent nemůže přepsat (sekce 2.2.3).

Kvalita testů (Q3–Q4)

Tabulka 3.4 shrnuje metriky kvality testů.

Tabulka 3.4: Metriky kvality testů Q3–Q4

Kód	Co měří	Jak se měří
Q3	Schopnost agentových testů detekovat chyby	Stryker (sekce 2.1.2) s konfigurací <code>--mutate 'src/**/*.ts'</code> a mutátory pro TypeScript (conditional, arithmetic, string, logical operators) systematicky zavádí drobné změny do zdrojového kódu. Vstup: agentovy testy + zdrojový kód. Výstup: procento zabitých mutantů.
Q4	Pokrytí acceptance criteria agentovými testy	Specifikace obsahuje 25 acceptance criteria. Hodnocení LLM-as-judge (sekce 3.2.4) určí, kolik z nich má odpovídající test v agentově test suite. Škála N/25.

Poznámka: Na rozdíl od Q2 (funguje implementace?) se Q4 ptá, jestli agent *testoval všechno co měl*.

Kvalita kódu (Q5–Q8)

Tabulka 3.5 shrnuje metriky kvality kódu.

Tabulka 3.5: Metriky kvality kódu Q5–Q8

Kód	Co měří	Jak se měří
Q5	Lint warnings + errors	ESLint s fixní konfigurací (recommended + strict TypeScript rules). Konfigurace je součástí experimentální infrastruktury, ne agentova repo; agent ji nemůže měnit. Výstup: celkový počet warnings + errors.
Q6	Typové chyby	<code>tsc --noEmit</code> ve strict mode. Doplnuje počet explicitních <code>any</code> ve zdrojových souborech (grep v <code>src/**/*.ts</code>) jako proxy pro obcházení typového systému.
Q7	Cyklomatická složitost	ESLint <code>complexity</code> rule reportuje funkce překračující práh. Výstup: maximální složitost přes všechny funkce.
Q8	Designová kvalita kódu	LLM-as-judge (sekce 3.2.4) hodnotí pojmenování, oddělení zodpovědností, idiomatický TypeScript, kvalitu dokumentace a zbytečnou komplexitu.

Poznámka: Práh ≤ 10 per funkci pro Q7 vychází z McCabe (sekce 2.1.2). Celkové skóre Q8 je *minimum* pěti dimenzí, nikoliv průměr. Volba minima je záměrná: jeden slabý rozměr má stáhnout celkový výsledek dolů, aby slabiny nebyly maskované silnými dimenzemi.

3.2.3 Metriky zdrojů (E1–E3)

Metriky zdrojů zachycují vstupy spotřebované při vývoji. Celá skupina (E1–E3) je homogenně deskriptivní: žádná z metrik nemá exit práh; hodnoty zaznamenáváme pro porovnání nákladů a stability runtime mezi iteracemi.

Tabulka 3.6: Metriky zdrojů E1–E3

Kód	Co měří	Jak se měří
E1	Spotřeba tokenů	Tři složky z exportu session v tisících: maximální vstup na jednom kroku, součet výstupních tokenů a součet cache tokenů. Motivace: Gloaguen et al. (Gloaguen et al., 2026) ukazují, že context files zvyšují inference cost o více než 20 %.
E2	Trvání běhu	Wall-clock time v minutách od spuštění agenta po poslední commit.
E3	Počet kompakcí	Počet kompakcí kontextu během běhu, čtený přímo z OpenCode DB (pole <code>time_compacting</code>). Kompakce indikuje, že agent vyčerpал kontextové okno a runtime musel obsah zhustit. Deskriptivní indikátor stability runtime; metrika nemá exit práh a nulová hodnota potvrzuje, že kontext zůstal celistvý po celou dobu běhu.

Se dvěma běhy per variaci jsou tyto hodnoty deskriptivní, ne inferenční.

3.2.4 Aplikace LLM-as-judge

Pět metrik (P6, P7, P8, Q4, Q8) hodnotí vlastnosti které nelze extrahovat automatizovaným nástrojem. Pro jejich hodnocení používáme metodu LLM-as-judge (sekce 2.2.3).

Volba modelu. Jako judge používáme GLM-5 (Zhipu AI). Agentní běhy provádí model MiniMax-M2.5; judge je tedy z jiné modelové rodiny. Důvod: model má tendenci hodnotit výstupy svého vlastního druhu lépe, i když jsou objektivně slabší (self-preference bias (Pannickssery et al., 2024)). Volba judge z jiné modelové rodiny než agent tento bias snižuje (Verga et al., 2024).

Škála. Hodnocení probíhá na škále 1–3 per dimenze: 1 = nevyhovující, 2 = přijatelné, 3 = dobré. Jemnější škály (1–5, 1–10) produkují při malém počtu hodnocených artefaktů nižší shodu mezi hodnotiteli (Zheng et al., 2023); třístupňová škála je pro tento rozsah spolehlivější.

Rubrika. Judge dostane artefakty agentova běhu (zdrojový kód, commit messages, issue a PR descriptions) spolu s rubrikou obsahující popis každého stupně. Dimenze per metriku jsou popsány při definici metrik (P6–P8 v sekci 3.2.1, Q4 a Q8 v sekci 3.2.2). Kompletní formulace rubrik a promptu je v repozitáři práce. Prompt je fixní napříč všemi běhy, aby hodnocení bylo srovnatelné mezi iteracemi.

Interpretace výsledků. LLM-as-judge zde neslouží jako plně objektivní měření, ale jako strukturované posouzení vlastností, které nelze spolehlivě vyhodnotit deterministickým skriptem. Výsledky těchto metrik proto interpretujeme jako podpůrné judge-based indikátory a ne jako hlavní důkazní osu případové studie.

3.3 Experimentální design

Sekce 3.1 představila výzkumnou strategii, zvolený projekt, fáze případové studie a strukturu iterativního cyklu. Tato sekce navazuje operačním detailem: jaké proměnné držíme fixní mezi běhy (sekce 3.3.1), jak jeden běh zaznamenáváme a vyhodnocujeme (sekce 3.3.2) a jakou procedurou diagnostikujeme a upravujeme instrukce (sekce 3.3.3).

3.3.1 Fixní proměnné

Aby bylo možné měřit vliv instrukcí, musí být všechno ostatní co nejstabilnější. Hlavní měněnou proměnnou mezi běhy je obsah souboru `AGENTS.md`, tedy procedurálních instrukcí, které definují pracovní postup, omezení a quality gates. Tento design negarantuje dokonalou izolaci všech vlivů; jeho cílem je omezit zjevné vedlejší vstupy a udělat jejich případný vliv auditovatelný. Na rozdíl od generických context files, které typicky popisují adresářovou strukturu a build příkazy (Gloaguen et al., 2026), jde o fokusované instrukce s konkrétními procedurami (X. Li et al., 2026).

Fixní proměnné jsou:

Prázdné GitHub repo obsahuje pouze `AGENTS.md` a konfiguraci agenta. Žádný existující kód, testy ani `package.json`. Agent musí inicializovat projekt a zvolit strukturu sám.

Specifikace projektu (popsaná v sekci 3.1.2) zůstává napříč všemi běhy beze změny a je dodávána agentovi v GitHub Issue #1 jako hlavní věcný vstup vedle instrukcí.

Model MiniMax-M2.5 byl zvolen jako model schopný autonomně dokončit zadaný úkol. Cílem práce je vyhodnotit sadu metrik a iterativní postup, ne srovnávat výkon modelů; volba konkrétního modelu je v tomto ohledu sekundární. MiniMax-M2.5 je zároveň z jiné modelové rodiny než judge (GLM-5, Zhipu AI), čímž se eliminuje self-preference bias (sekce 2.2.3). Teplota modelu nebyla mezi běhy explicitně nastavena. Agent běžel s výchozí konfigurací poskytovatele. Dopad této volby na variabilitu výsledků diskutuje sekce 5.3.

Běhové prostředí (Docker container) zajišťuje, že agent nemá přístup ke globální konfiguraci nástroje, MCP serverům ani rozšířením nainstalovaným na hostitelském systému. Izolace tím významně omezuje vliv vedlejších vstupů z prostředí, ale nevyklučuje všechny zdroje variability. Sdílený autentizační svazek zajišťuje konzistentní přístup k API napříč běhy.

Harness OpenCode je CLI nástroj, který orchestruje běh agenta: spravuje session, zprostředkovává volání modelu, zpřístupňuje nástroje (čtení a zápis souborů, shell, vyhledávání) a zaznamenává transcript. Harness sám není měřeným chováním; měříme to, co agent přes něj udělá. Verze nástroje je pinovaná v Docker image, konfigurace agenta (povolené nástroje, MCP servery) je součástí auditovatelného balíku v repozitáři práce. Volba OpenCode je dána open-source licencí (umožňuje pinování verze a záznam interní telemetrie), strojově čitelným exportem session a přístupem do interní databáze, ze které čerpá metrika E3 (počet kompakcí

kontextu). Vlastnosti harness, které ovlivňují měření, jsou diskutovány u jednotlivých metrik: tichý retry po selhání API (sekce 5.3) a automatická kompakce kontextu (metrika E3).

System prompt agenta (`build.md`) nahrazuje výchozí system prompt nástroje OpenCode, jehož instrukce o verzování konfliktovaly s procesními požadavky případové studie. Vlastní system prompt obsahuje pouze obecné kódové konvence (styl, bezpečnost, zákaz komentářů) a odkaz na `AGENTS.md`. Veškeré procesní instrukce jsou výhradně v `AGENTS.md`, aby bylo možné změny procesního chování přisuzovat této vrstvě obhajitelněji než při jejich rozptýlení mezi více vstupů.

3.3.2 Záznam a vyhodnocení běhu

Každý běh produkuje strukturovaný balík záznamů, který umožňuje zpětně rekonstruovat naměřené hodnoty i chování agenta.

Sběr metrik. Po skončení běhu se spouštějí dva skripty. `new-run.ts` exportuje session do `transcript.json` (úplný záznam tool calls, vstupů a výstupů agenta). `analyze-run.ts` extrahuje deterministické metriky z git historie, GitHub API a transcriptu (P1–P5, Q1–Q3, Q5–Q7, E1–E3). Skript `judge.ts` spouští LLM-as-judge (GLM-5) pro judge-based metriky (P6–P8, Q4, Q8) podle rubrik uložených v repozitáři práce. Souhrnným výstupem je `FINDINGS.md` s tabulkou metrik a faktickým popisem chování agenta.

Auditovatelný balík. Každý běh má vlastní adresář `experiments/runs/pilot-rN/` obsahující transcript, naměřené metriky a `FINDINGS.md`. K iteraci jako celku patří navíc `DIAGNOSIS.md` s analytickým výstupem (sekce 3.3.3) a záznam v changelogu zdůvodňující úpravu `AGENTS.md` pro další běh. Tabulka 3.7 shrnuje strukturu balíku.

Tabulka 3.7: Záznamy pořízené pro jeden běh a jednu iteraci

Soubor	Obsah
<code>transcript.json</code>	úplný záznam session (tool calls, vstup, výstup)
<code>FINDINGS.md</code>	tabulka metrik a faktický popis chování
<code>DIAGNOSIS.md</code>	klasifikace příčin a návrh změn
<code>changelog/rN-rN+1.md</code>	zdůvodnění úprav <code>AGENTS.md</code> (bylo / je / pozorování / diagnóza / literatura)

Doplňující materiály jsou v repozitáři práce (<https://github.com/7onc3k/Bakalarka/tree/thesis-final>): kompletní operační procedura, šablony `FINDINGS.md` a `DIAGNOSIS.md`, všechny verze `AGENTS.md` včetně baseline a ablačních variant, plné rubriky a prompty LLM-as-judge, transcripty session, surová výstupní data nástrojů a specifikace projektu v plném znění. Stav repozitáře k datu odevzdání je zafixován gitovým tagem `thesis-final`.

3.3.3 Diagnostika a úprava instrukcí

Tato sekce popisuje analytickou část iterace: jak z naměřených hodnot odvodíme příčinu selhání (diagnóza) a jak z ní vychází konkrétní úprava `AGENTS.md` (úprava). Vstupem je `FINDINGS.md` z předchozího běhu (sekce 3.3.2), výstupem `DIAGNOSIS.md`, upravená `AGENTS.md` pro příští běh a záznam v changelogu.

Diagnostické rámce. Diagnóza identifikuje, kde a proč agent nedodržel očekávané chování. Opírá se o pět rámců, z nichž každý kladou jinou diagnostickou otázku (tabulka 3.8).

Tabulka 3.8: Diagnostické rámce a otázky, které kladou

Rámec	Diagnostická otázka
Mao et al. (2025)	Jsou komponenty instrukce přítomné a ve správném pořadí?
Hassan et al. (2025)	Vyvažuje obsah cíl (briefing), postup (loop) a omezení (mentor)?
Razavi and Fard (2025), Breunig (2025)	Proč agent instrukci opakovaně ignoruje a jak ji přestrukturovat?
Lulla et al. (2026)	Které typy obsahu reálně přispívají k chování? *
Filter X. Li et al. (2026)	<i>Kdyby tento řádek chyběl, udělal by agent neočividnou chybu?</i>

* Lulla měří přítomnost celého souboru, ne jednotlivé typy obsahu; rámec proto používáme interpretativně.

Postup diagnózy. Z tabulky metrik se vyberou nesplněné nebo hraniční hodnoty, dohledají se faktické projevy ve `FINDINGS.md` a v artefaktech (git log, issues, PR, transcript), aby bylo možné odlišit jednorázové selhání běhu od opakujícího se problému instrukcí. Identifikovaná příčina se klasifikuje podle tabulky 3.9, která mapuje typ příčiny na primární rámec a typ opravy.

Tabulka 3.9: Mapování klasifikace příčiny na diagnostický rámec a typ opravy

Klasifikace příčiny	Primární rámec	Typ opravy
Chybějící / příliš obecná složka	Mao	Doplnění / zpřesnění
Nevyvážený obsah	Hassan	Přestrukturování
Opakovaně ignorovaná instrukce	Razavi, Breunig	Přestrukturování + verifikace
Redundantní obsah	Lulla + filter Li	Odebrání

Hranice mezi typy příčin jsou v praxi neostře a klasifikace vyžaduje úsudek; rámce slouží jako heuristické vodítko, ne algoritmus. Roli AI asistence při této analýze popisuje sekce 3.5.

Úprava. Na základě diagnózy se upraví `AGENTS.md`. Každá změna odpovídá jednomu identifikovanému problému, je podložena citací a zaznamenána v changelogu ve formátu *bylo / je / pozorování / diagnóza / literatura*, aby bylo zpětně dohledatelné, proč daná úprava vznikla a co měla opravit. Přestrukturování instrukcí má přednost před jejich rozšiřováním: redundantní obsah zvyšuje inference cost bez přínosu k úspěšnosti (Gloaguen et al., 2026) a fokusované instrukce překonávají vyčerpávající dokumentaci (X. Li et al., 2026).

Iterativní úpravu instrukcí by bylo možné automatizovat přístupy typu *automatic prompt optimization* (APO) (Schnabel & Neville, 2024), kde smyčka *score* → *synthesize* formálně odpovídá našemu cyklu *měření* → *diagnóza* → *úprava*; reprezentanty jsou například PromptWi-

zard (Agarwal et al., 2024) a Prompt Alchemy (Ye et al., 2025). Volíme ruční postup s nástrojem Claude Code jako analytickým asistentem (popis využití v příloze A): vícerozměrné hodnocení P/Q/E nelze sjednotit do jediné loss funkce, měřítko studie (desítky běhů) neodpovídá řádu APO (tisíce evaluací) a cílem je vysvětlit, *proč* instrukce selhává a *jak* se opravuje, ne jen najít kombinaci s nejvyšším score.

3.4 Přehledová tabulka

Tabulka 3.10 shrnuje všechny metriky na jednom místě. Sběr má dvě podoby: deterministické metriky (sloupec *Nástroj* ukazuje konkrétní automatizovaný nástroj) a judge-based metriky (sloupec *Nástroj* obsahuje LLM-as-judge). Prahové požadavky ve sloupci *Exit kritérium* popisují striktní ideál pro procesní metriky (P1–P8) a produktové metriky (Q1–Q8); deskriptivní indikátory bez prahu (celá zdrojová skupina E1–E3) mají v tomto sloupci pomlčku a slouží jen k porovnání mezi běhy. Pro praktické rozlišení mezi běhy se v analýze používají též operační prahy popsané v sekci 3.1.3.

Tabulka 3.10: Sada metrik: kód, měřená vlastnost, nástroj a exit kritérium

Kód	Metrika	Nástroj	Exit kritérium
<i>Procesní (P): jak agent pracuje</i>			
P1	Issues before code	git, GitHub API	pass
P2	Branch per issue	git, GitHub API	pass
P3	Test-first commits	git log	pass
P4	PRs linked to issues	GitHub API	pass
P5	No existing test modifications	git diff	pass
P6	Commit message quality	LLM-as-judge (GLM-5)	≥ 2/3
P7	Issue description quality	LLM-as-judge (GLM-5)	≥ 2/3
P8	PR description quality	LLM-as-judge (GLM-5)	≥ 2/3
<i>Produktové (Q): co agent vyrobil</i>			
Q1	API contract match	tsc (import + typecheck)	match
Q2	Referenční test pass rate	Vitest (42 testů)	42/42
Q3	Mutation score	Stryker	≥ 70 %
Q4	AC coverage agentových testů (25 krit.)	LLM-as-judge (GLM-5)	25/25
Q5	Lint warnings	ESLint	0
Q6	Typecheck errors	tsc --noEmit	0
Q7	Cyklomatická složitost	ESLint (complexity)	≤ 10/fn
Q8	Design quality	LLM-as-judge (GLM-5)	≥ 2/3
<i>Zdroje (E): za jakou cenu</i>			
E1	Vstup / výstup / Σ cache (tis.) z exportu	OpenCode export	—
E2	Trvání (minuty)	session timestamps	—
E3	Počet kompakcí kontextu	OpenCode DB (time_compacting)	—

3.5 Omezení a validita

Případová studie nese omezení daná zvolenou metodikou. Tato sekce deklaruje hrozby validity, které jsou s designem spojené a známe je předem; analýza vychází z kategorizace Runeson

a Höst (Runeson & Höst, 2009). Jak se tyto hrozby skutečně projeví v naměřených datech, diskutuje sekce 5.3.

Konstruktová validita (měříme to, co chceme měřit?). Jednotlivé metriky vychází z existující teorie: mutation testing (Papadakis et al., 2019), cyklomatická složitost (McCabe, 1976), taxonomie procesních a produktových metrik (Fenton & Bieman, 2014). Konkrétní kombinace metrik do sady a volba exit kritérií jsou autorské a představují návrh, ne ověřený standard. Pět metrik (P6, P7, P8, Q4, Q8) obsahuje subjektivní složku (LLM-as-judge). Tyto metriky proto slouží jako podpůrné judge-based indikátory, zatímco hlavní opora vyhodnocení stojí na deterministických metrikách a auditovatelných artefaktech. Spolehlivost LLM-as-judge nebyla validována proti lidskému hodnocení (např. Cohenovým κ) z důvodu časových omezení. U metriky Q4 byly historicky uložené judge běhy exportovány ve 24bodovém formátu; chybějící bod AC25 (*custom holiday calendar*) byl při finalizaci práce dopočítán manuálně. Správnost skriptu `analyze-run.ts` pro metriky P1–P5 byla ověřena manuálním srovnáním výstupů s raw daty z git logu a GitHub API pro běhy r4 a r5. Žádná diskrepance nebyla identifikována.

Interní validita (jsou závěry podložené daty?). Hlavní hrozba je nedeterminismus LLM: stejné instrukce mohou při různých bězích dát různé výsledky (Razavi & Fard, 2025). Pilotní běhy r1–r5 probíhaly jednorázově, a tato hrozba v nich není mitigována; jejich roli je proto třeba číst jako formativní (iterativní návrh instrukcí), ne jako konfirmační měření efektu. Mitigace se uplatňuje až v ablacích, kde každou variaci spouštíme dvakrát: shoda obou běhů ve směru efektu zvyšuje důvěru, že nejde o náhodu, byť ani dva běhy neposkytují statistickou sílu. Další hrozbou je confirmation bias: autor navrhuje instrukce i evaluuje výsledky. Tuto hrozbu oslabují deterministické metriky (P1, Q1–Q3, Q5–Q7), které nepodléhají subjektivnímu posouzení, a volba judge modelu z jiné rodiny. Mitigací je také auditovatelný řetězec rozhodnutí (changelog per iterace, korekční poznámky v `FINDINGS.md`) a oddělení měřicí infrastruktury od agentova prostředí.

Externí validita (lze zobecnit?). Případová studie na jednom projektu, jednom modelu a jednom agentním nástroji neumožňuje statistickou generalizaci. Yin (R. K. Yin, 2018) pro tento typ výzkumu rozlišuje analytickou generalizaci: z jednoho případu lze ukázat principy, ne statistické zákonitosti. Systém upomínek má deterministickou logiku a výsledky nemusí platit pro projekty s uživatelským rozhraním, strojovým učením nebo nedeterministickými výstupy. Další ověřování má proto dvě osy: replikaci v programování na jiných modelech, nástrojích a projektech a ověření mimo programování, kde nemusí existovat deterministická zpětná vazba. Přenositelné jsou metriky a postup (kdokoliv je může použít na svém projektu), konkrétní naměřené hodnoty a instrukce platí pro tuto studii.

Výzkum nezahrnuje lidské účastníky. Veškerá data (git log, metriky, session transcripts) jsou generována experimentální infrastrukturou. Zdrojový kód a experimentální data jsou veřejně dostupná.

4. Praktická část

Tato kapitola popisuje provedení případové studie. Specifikace projektu, referenční testy a experimentální setup jsou popsány v kapitole 3. Struktura kapitoly sleduje chronologii studie: výchozí instrukce použité v prvním běhu, pak pilotní fáze kde se instrukce opakovaně upravují na základě naměřených metrik, a nakonec ablace, kde z fungující sady systematicky odebíráme jednotlivé části a měříme dopad.

4.1 Konstrukce baseline instrukcí

Baseline verze `AGENTS.md` byla konstruována čistě z literatury. Cílem bylo vytvořit výchozí bod jehož každá komponenta má opodstatnění v empirických zjištěních. Literatura společně naznačuje, že výchozí `AGENTS.md` musí spojit tři vrstvy: jasné vymezení úkolu, stručný procedurální pracovní postup a explicitní verifikační kroky kvality. Mao et al. (Mao et al., 2025) poskytují kostru komponent, Li et al. (X. Li et al., 2026) ukazují vyšší účinnost procedurálních instrukcí než popisné dokumentace, Breunig et al. (Breunig, 2025) motivují převod obecných pravidel na konkrétní akce a Lulla et al. (Lulla et al., 2026) varují před přidáváním obsahu bez jasné hodnoty. Baseline proto nevznikla jako vyčerpávající manuál, ale jako kompaktní soubor instrukcí, jehož sekce se mapují na konkrétní měřené dimenze.

Konstrukce proběhla ve dvou krocích: adaptace sedmi typů komponent identifikovaných Mao et al. (Mao et al., 2025) na konkrétní sekce `AGENTS.md` (Role, Goal, Specification, Environment, Process, Package Quality, Constraints) a mapování tří měřených dimenzí (sekce 3.2) na obsah těchto sekcí.

Sekce Package Quality byla přidána proto, že udržitelnost a design nejsou ve specifikaci projektu explicitně vynutitelné, ale v naší sadě metrik je chceme sledovat. Constraints obsahuje zákazy vzorců, které jsme ve fázi přípravy považovali za rizikové pro autonomního agenta (slučování více issues do jedné větve, modifikace existujících testů, přepis git historie). Výsledný dokument má 53 řádků a ~350 slov; tabulka 4.1 ukazuje mapování jednotlivých sekcí na metriky a obrázek 4.1 uvádí kompletní znění instrukcí.

Tabulka 4.1: Mapování sekcí baseline `AGENTS.md` na metriky

Cíl	Sekce <code>AGENTS.md</code>	Metriky
Spec-first	Specification, Process krok 2	P1
Strukturovaný pracovní postup	Process kroky 1–4	P2, P3, P4
TDD	Process krok 2, Constraints	P3, P5
Modulární kód	Package Quality	Q5, Q7, Q8
API kompatibilita	Specification (API Contract)	Q1
Funkční korektnost	Specification (AC)	Q2, Q4


```

Baseline AGENTS.md

# AGENTS.md --- Dunning System

## Role

You are a senior TypeScript developer building a production-grade npm package. You prioritize clean architecture,
spec compliance, and test quality.

## Goal

Implement the dunning system (billing reminder state machine) specified in GitHub Issue #1. Deliver a modular,
documented, publishable TypeScript package.

## Specification

Read Issue #1 completely before writing any code. It contains:
- **Acceptance Criteria** --- every one must be satisfied and tested
- **API Contract** --- exact TypeScript signatures to export
- **Domain Glossary** --- use these terms in code and documentation
- **Out of Scope** --- do not implement

All design decisions must trace back to the spec. Do not invent requirements.

## Environment

- Runtime: Node.js, TypeScript (strict mode, ESM)
- Test runner: Vitest
- Build: tsc
- Tools: GitHub CLI (`gh`), Git

## Process

Decompose the spec into focused GitHub issues before writing code. Work through them sequentially:

1. Create issues --- one concern per issue, starting with project setup
2. For each issue: branch from main, write tests first, implement, PR with "Closes #N", merge
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Run `tsc --noEmit` before every PR --- no type errors allowed

## Package Quality

Structure the code as a production-grade npm package:

- **Modular architecture** --- separate files for types, business logic, utilities, and public API re-exports. No
single-file implementations.
- **Strict TypeScript** --- no `any`, explicit return types on public API, `readonly` where applicable.
- **Documentation** --- JSDoc on all exported functions and types. Inline comments for non-obvious domain logic
(e.g., business day calculations, escalation thresholds).
- **Clean API surface** --- `src/index.ts` re-exports only the public API. Internal modules are not exposed.

## Constraints

- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the spec --- fix the code, not the test.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest, typescript).

```

Obrázek 4.1: Baseline AGENTS.md (pilot-r1): kompletní znění instrukcí použitých jako výchozí bod pilotních iterací.

4.2 Pilotní fáze

Diagnóza každé iterace stojí na jednom běhu, takže opakující se selhání napříč iteracemi mají vyšší výpovědní hodnotu než jednorázová; tuto opatrnost atributu doplňuje až ablační fáze dvojicí běhů per variace (sekce 3.5). Iterativní cyklus zároveň nepředpokládá monotónní zlepšování. Pokud iterace vede k regresi, další iterace vychází z předchozí úspěšné verze instrukcí, ne z regresní (Peppers et al., 2008). Souhrnný přehled všech běhů obsahuje sekce 4.4.

4.2.1 Pilot-r1: baseline

První běh s baseline instrukcemi (verze v repozitáři práce), agent MiniMax-M2.5 přes Open-Code, proběhl v jedné session bez restartů. Tabulka 4.2 shrnuje naměřené metriky.

Tabulka 4.2: Pilot-r1: naměřené metriky

Kód	Metrika	Hodnota	Kritérium	Splněno?
P1	Issues before code	✓	splněno	✓
P2	Branch per issue	×	branches $\geq$ issues	×
P3	Test-first commits	×	test: před feat:	×
P4	PRs linked to issues	✓	všechny PR	✓
P5	Testy nezměněny	×	0 změn	×
P6	Commit msg quality	n/a	$\geq 2/3$	n/a
P7	Issue quality	3/3	$\geq 2/3$	✓
P8	PR quality	2/3	$\geq 2/3$	✓
Q1	API contract match	match	match	✓
Q2	Ref. test pass rate	39/42	42/42	×
Q3	Mutation score	84 %	≥ 70 %	✓
Q4	AC coverage (agentovy testy)	23/25	25/25	×
Q5	Lint warnings	2	0	×
Q6	Typecheck errors	0	0	✓
Q7	Complexity violations	2 viol.	0 viol.	×
Q8	Design quality	1/3	$\geq 2/3$	×
E1	Vstup / výstup / Σ cache (tis.)	115 / 60 / 11528	n/a	záznam
E2	Trvání	33 min	n/a	záznam
E3	Počet kompakcí	0	n/a	záznam

Z binárních pass/fail kritérií prošla v r1 jen procesní složka (P1, P4) a minimální produktová sada (Q1, Q6); ostatní kritéria včetně Q2, Q5 a Q7 agent nesplnil.

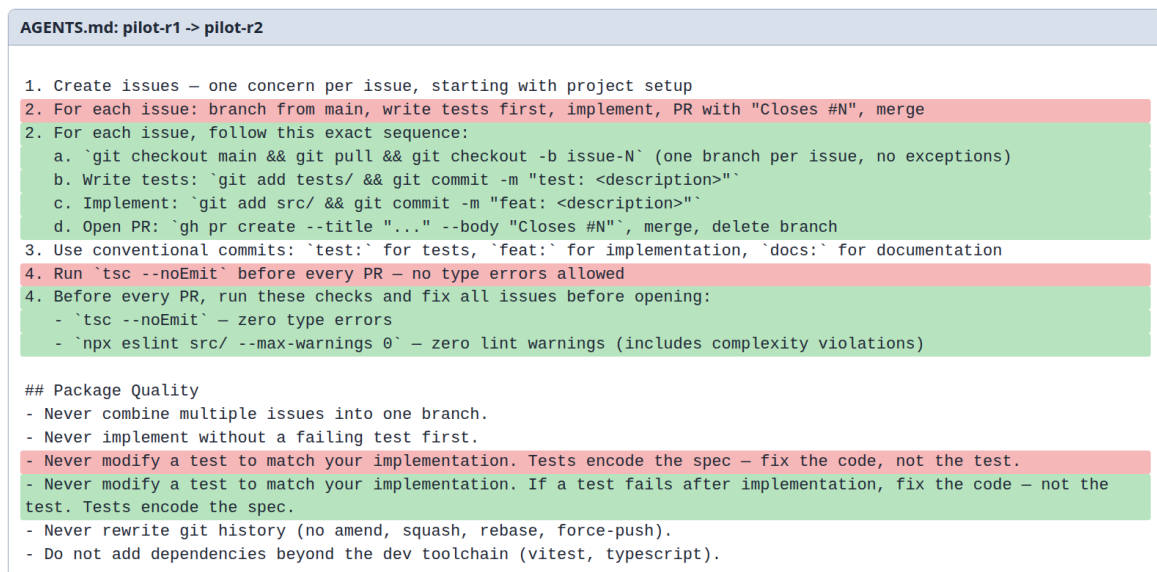
Diagnostika.

Agent vytvořil 11 issues hromadně na začátku běhu (před prvním commitem do `src/`) a rozložil implementaci do 4 větví, každá s vlastním pull requestem. Větve agregovaly související issues,

takže P2 (branch per issue) selhala v poměru 4 větve na 11 issues. Napříč všemi 4 větvemi předcházela implementace testů (P3 (test-first commits)); agent vlastní testy psal až po kódu a v jedné větvi navíc upravil existující test (P5 (no existing test modifications)). Z referenčních testů selhaly 3 z 42 (Q2 (referenční test pass rate)): jedno selhání pause/resume na zachování uplynulého času a dvě v manuálním přechodu mezi stavy. Q8 (design quality) = 1/3 kvůli chybějící JSDoc dokumentaci na veřejném API. Vlastních testů napsal agent 59 a všechny procházely. Strukturu kódu zvolil modulární (`types.ts`, `businessDays.ts`, `dunning.ts`, `index.ts`), což odpovídá požadavkům sekce Package Quality v instrukcích.

Procesní selhání P2, P3 a P5 sdílí stejný vzor: instrukce uvedla, *co* má platit (jeden issue ve vlastní větvi, testy před kódem, existující test se neupravuje), bez konkrétního příkazu nebo kontroly. Agent v každém z nich zvolil rychlejší alternativu, která literu pravidla neporušila. Q7 (složitost kódu) (2 funkce nad prahem) sledovala stejný vzor: „Package Quality“ požadovala modulární kód, ale neenforcovala kontrolu složitosti. Selhání referenčních testů jsou jiného typu, jde o implementační rozhodnutí v pause/resume logice a výpočtech v pracovních dnech, na která úroveň instrukcí v baseline nedosahuje.

Pro r2 doplňujeme procedurální páku ke každému selhanému procesnímu pravidlu (příkaz pro vytvoření větve, oddělené staging testů a kódu, korektivní pravidlo pro selhávající test) a kontrolu lintu před otevřením pull requestu. Konkrétní text změn ukazuje obrázek 4.2.



Obrázek 4.2: Vizuální diff změn AGENTS.md mezi pilot-r1 a pilot-r2

4.2.2 Pilot-r2

Druhý běh ověřoval čtyři změny instrukcí. Tabulka 4.3 ukazuje metriky kde došlo ke změně.

Tabulka 4.3: Pilot-r2: metriky se změnou oproti r1

Kód	Metrika	r1	r2	Trend
P5	Testy nezměněny	×	✓	zlepšení
Q2	Ref. test pass rate	39/42	32/42	regrese
Q3	Mutation score	84 %	68 %	regrese
Q5	Lint warnings	2	1	zlepšení
Q7	Complexity violations	2 viol.	0	zlepšení
Q8	Design quality	1/3	1/3	stabilní
E1	Vstup / výstup / Σ cache (tis.)	115 / 60 / 11528	76 / 41 / 3196	záznam
E2	Trvání	33 min	37 min	záznam

Diagnostika.

P5 a Q7 byly opraveny: existující testy zůstaly nezměněné a žádná funkce nepřekročila práh složitosti. P2 a P3 ale přetrvaly v jiné podobě než v r1. Agent vytvořil 7 issues a 3 větve (jedna nesla 4 issues), tedy stejný vzor agregace jako v r1, jen v menším měřítku. P3 selhala formálně přesně opačně: všechny tři commity dostaly prefix `test:` a obsahovaly současně testy i implementaci, takže metrika „`test:` commit před `feat:`“ nemohla projít, protože žádný `feat:` commit nevznikl. Q8 = 1/3 zůstala beze změny: deklarativní požadavek na JSDoc agent ignoroval. Q2 a Q3 se zhoršily (32/42, resp. 68 %); selhaly zejména testy eskalace v pracovních dnech. Vlastních testů napsal agent 70 (oproti 59 v r1), ale nižší kvality.

V obou procesních pravidlech splnil agent literu nového procedurálního příkazu, ale ne jeho záměr. Procedura vyžadovala `git checkout -b issue-N` bez explicitního pravidla „jeden issue $\Rightarrow$ jedna větev“, a oddělené staging testů a kódu bez podmínky, že každý commit nese jen jednu z obou stran. Pokles kvality testů ukazuje analogický posun na produktové straně: agent patrně napsal celou test suite hromadně ze specifikace místo postupného test-fix cyklu, takže více testů kontrolovalo méně mutantů. E1 klesla na promptové i výstupní straně, ale pro rozhodnutí o další úpravě byla podstatná hlavně regrese v testování a dokumentaci.

Pro r3 přesouváme tři instrukce z roviny příkazu do verifikačního kroku: explicitní listing otevřených issues před výběrem práce, kontrolní `git log` mezi commity testů a kódu a přesun požadavku na JSDoc do pre-PR checklistu. Konkrétní text změn ukazuje obrázek 4.3.

AGENTS.md: pilot-r2 -> pilot-r3				
1.	Create issues – one concern per issue, starting with project setup			
2.	For each issue, follow this exact sequence:			
a.	<code>`git checkout main && git pull && git checkout -b issue-N`</code> (one branch per issue, no exceptions)			
a.	Pick one open issue: <code>`gh issue list --state open`</code> , then <code>`git checkout main && git pull && git checkout -b issue-N`</code>			
b.	Write tests: <code>`git add tests/ && git commit -m "test: <description>"`</code>			
c.	Implement: <code>`git add src/ && git commit -m "feat: <description>"`</code>			
d.	Open PR: <code>`gh pr create --title "... " --body "Closes #N"`</code> , merge, delete branch			
c.	Verify: run <code>`git log --oneline -3`</code> and confirm the test: commit exists before proceeding			
d.	Implement: <code>`git add src/ && git commit -m "feat: <description>"`</code>			
e.	Open PR: <code>`gh pr create --title "... " --body "Closes #N"`</code> , merge, delete branch			
3.	Use conventional commits: <code>`test:`</code> for tests, <code>`feat:`</code> for implementation, <code>`docs:`</code> for documentation			
4.	Before every PR, run these checks and fix all issues before opening:			
-	Modular architecture – separate files for types, business logic, utilities, and public API re-exports. No single-file implementations.			
-	Strict TypeScript – no <code>`any`</code> , explicit return types on public API, <code>`readonly`</code> where applicable.			
-	Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds).			
-	Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds). This is enforced in Constraints.			
-	Clean API surface – <code>`src/index.ts`</code> re-exports only the public API. Internal modules are not exposed.			
-	Never implement without a failing test first.			
-	Never modify a test to match your implementation. If a test fails after implementation, fix the code – not the test. Tests encode the spec.			
-	Before every PR, verify JSDoc on every exported function in <code>`src/index.ts`</code> . If any export lacks JSDoc, add it before opening the PR.			
-	Never rewrite git history (no amend, squash, rebase, force-push).			
-	Do not add dependencies beyond the dev toolchain (vitest, typescript).			

Obrázek 4.3: Vizuální diff změn AGENTS.md mezi pilot-r2 a pilot-r3

4.2.3 Pilot-r3

Třetí běh ověřoval tři změny instrukcí z r2: verifikační příkaz po commitu s testy, výběr issue přes `gh issue list` a přesun požadavku na JSDoc do verifikačního kroku před pull requestem. Tabulka 4.4 ukazuje metriky s výraznou změnou.

Tabulka 4.4: Pilot-r3: metriky se změnou oproti r2

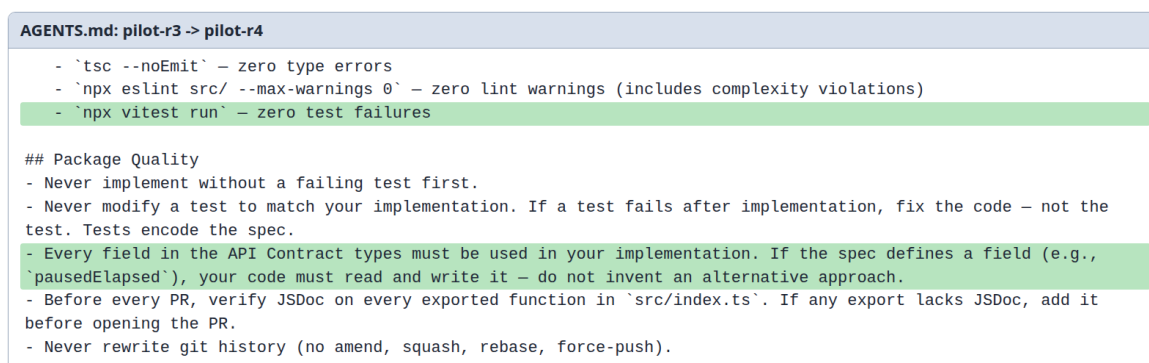
Kód	Metrika	r2	r3	Trend
P2	Branch per issue	×	✓	zlepšení
P3	Test-first commits	×	✓	zlepšení
P6	Commit msg quality	2/3	3/3	zlepšení
P7	Issue quality	2/3	3/3	zlepšení
Q2	Ref. test pass rate	32/42	41/42	zlepšení
Q3	Mutation score	68 %	71 %	zlepšení
Q8	Design quality	1/3	3/3	zlepšení
E1	Vstup / výstup / Σ cache (tis.)	76 / 41 / 3196	62 / 30 / 4770	záznam
E2	Trvání	37 min	25 min	záznam
E3	Počet kompakcí	0	0	záznam

Diagnostika.

Agent poprvé splnil celý procesní checklist P1–P5 = 5/5: vytvořil issues před kódem, každý issue dostal vlastní branch, test commit předcházel kódu, pull requesty obsahovaly `Closes #N` a žádný test nebyl dodatečně upraven. Q8 = 3/3 odpovídá doplnění JSDoc dokumentace po přesunu požadavku do verifikačního kroku. Všechny tři změny tedy zabraly.

Q2 = 41/42: zbývá jedna chyba v implementaci, a odpovídá prvnímu ze dvou implicitních pravidel popsanych v sekci 3.1.2. Specifikace definuje chování pause/resume jednoznačně: acceptance criteria vyžadují “elapsed time are preserved” při pauzování a “timeout resumes from where it left off” při obnovení. API kontrakt navíc poskytuje pole `pausedElapsed` s komentářem “business days elapsed before pause”. Agent přesto zvolil vlastní přístup (posunutí `stateEnteredAt` zpět v čase) místo použití pole z kontraktu. Agentův přístup fungoval pro jedno pozastavení, ale selhal při opakovaném pause/resume. Spec není nejednoznačná. Agent učinil implementační rozhodnutí navzdory explicitnímu poli v kontraktu.

Pro r4 doplňujeme do pre-PR checklistu příkaz `npx vitest run`, aby agent před otevřením pull requestu ověřil průchod vlastních testů, a do Constraints pravidlo: každé pole v API kontraktu musí být v implementaci použito (Breunig, 2025). Konkrétní text změn ukazuje obrázek 4.4.



Obrázek 4.4: Vizuální diff změn `AGENTS.md` mezi pilot-r3 a pilot-r4

4.2.4 Pilot-r4

Čtvrtý běh ověřoval dvě úpravy: přidání `npx vitest run` do pre-PR checklistu a pravidlo o dodržení API kontraktu (každé pole v typech musí být použito v implementaci). Tabulka 4.5 ukazuje metriky s nejvýraznější změnou.

Tabulka 4.5: Pilot-r4: metriky se změnou oproti r3

Kód	Metrika	r3	r4	Trend
P2	Branch per issue	✓	×	regrese
P5	Testy nezměněny	✓	×	regrese
Q2	Ref. test pass rate	41/42	39/42	regrese
Q3	Mutation score	71 %	n/a	n/a
Q8	Design quality	3/3	2/3	regrese
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	81 / 36 / 5996	záznam
E2	Trvání	25 min	26 min	záznam
E3	Počet kompakcí	0	0	záznam

Diagnostika.

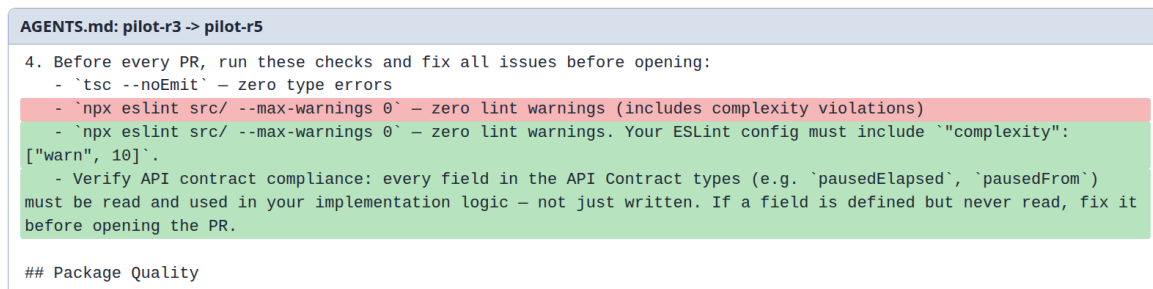
Agent vytvořil 7 issues ale zpracoval jen 5. Dva issues zůstaly bez branch (P2 = 4/6) a upravil dva vlastní testové soubory po implementaci (P5). Tabulka 4.5 ukazuje regrese i v Q2 a Q8.

Tato regrese ale nemá jediný zdroj: mezi běhy r3 a r4 se kromě instrukcí změnila i konfigurace modelu v Docker prostředí (sekce 5.3). Pokles proto nelze připsat výhradně dvěma novým řádkům v **Constraints**.

Q2 se zhoršilo z 41/42 na 39/42: k přetrvávajícímu elapsed time bugu přibýly dvě nové chyby (výpočet víkendů v eskalaci a boundary DUE_SOON přechodu) odpovídající druhému implicitnímu pravidlu domény (sekce 3.1.2). Pravidlo o dodržení API kontraktu nepomohlo. Agent pole `pausedElapsed` opět nepoužil. Agentův vlastní test navíc obsahoval timezone bug a přesto agent mergoval kód, přestože pre-PR checklist zahrnoval `npm vitest run`.

Deklarativní pravidlo v **Constraints** tedy zde nepřineslo zlepšení a instrukce na této úloze zůstávají citlivé na variabilitu mezi běhy.

Pro r5 vycházíme z r3 (ne z r4) a cílíme na dvě zbývající selhání verifikačními kroky. Pro Q2 doplňujeme do pre-PR checklistu ověření, že každé pole z typů v API kontraktu je v implementaci čteno i zapisováno; oproti r4, kde pravidlo bylo v **Constraints**, je verifikace v **Process** (Breunig, 2025). Pro Q5/Q7 přidáváme explicitní požadavek na ESLint config zahrnující `complexity: [warn, 10]`, protože v r3 agent spustil eslint s vlastním configem bez tohoto pravidla a kód považoval za hotový. Konkrétní text změn ukazuje obrázek 4.5.



Obrázek 4.5: Vizuální diff změn AGENTS.md mezi pilot-r3 a pilot-r5

4.2.5 Pilot-r5: verifikace kontraktu

Pátý běh ověřoval změny zavedené po r4: API contract verifikaci v pre-PR checklistu a ESLint complexity pravidlo.

Tabulka 4.6: Pilot-r5: metriky se změnou oproti r3

Kód	Metrika	r3	r5	Trend
P2	Branch per issue	✓	×	regrese
P3	Test-first commits	✓	×	regrese
P4	PRs linked	✓	×	regrese
Q2	Ref. test pass rate	41/42	38/42	regrese
Q5	Lint warnings	1	0	zlepšení
Q7	Complexity violations	1 viol.	0	zlepšení
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	65 / 23 / 3028	záznam
E2	Trvání	25 min	13 min	záznam
E3	Počet kompakcí	0	0	záznam

Diagnostika.

Instrukce r5 se lišily od r3 pouze ve dvou řádcích pre-PR checklistu (API contract verifikace a ESLint complexity config). Obě cílené opravy zabraly: $Q5 = 0$ a $Q7 = 0$. Agent konfiguroval ESLint s pravidlem `complexity` a rozložil funkci `processTick`. Opravu $Q5/Q7$ tak pravděpodobně přinesla jediná konkrétní instrukce: “your ESLint config must include `complexity: [warn, 10]`”.

Přesto agent ignoroval celý pracovní postup v Process: žádné issues, branches ani pull requesty. Analýza transcriptu ukazuje posloupnost: agent přečetl instrukce (zpráva 4: “I need to decompose the spec into focused GitHub issues”), v následujícím kroku však místo `gh issue create` použil interní plánovací nástroj (`todowrite`) a implementoval vše v jednom `feat:` commitu. Pre-PR checks (`tsc`, `eslint`) přitom dodržel. Agent tedy selektivně plnil instrukce: jednokrokové příkazy s okamžitým výstupem (“spust `eslint`”) ano, vícekrokový pracovní postup (“vytvoř issue, pak branch, pak test, pak implementuj, pak PR”) ne.

Srovnání r3, r4 a r5 ukazuje, že tento vzorec není náhodný. R3 agent dodržel celý pracovní postup ($P1-P5 = 5/5$). R4 dodržel částečně ($\sim 3/5$), vytvořil issues ale ne pro všechny branch. R5 pracovní postup ignoroval zcela ($\sim 1/5$). Instrukce se přitom mezi r3 a r5 liší minimálně (2 řádky v pre-PR checklistu). Data tak naznačují, že nedeterminismus modelu je pro dodržování vícekových sekvencí silnějším faktorem než drobné lokální změny v pre-PR checklistu.

Přechod k ablacím.

Pilotní baseline pro ablace zůstává r3 (viz srovnání r3–r5 výše). Ablace testují, které složky fungujících instrukcí ke zlepšení skutečně přispěly. Interpretaci pilotních pozorování ve vztahu k cílům práce uvádí kapitola 5.

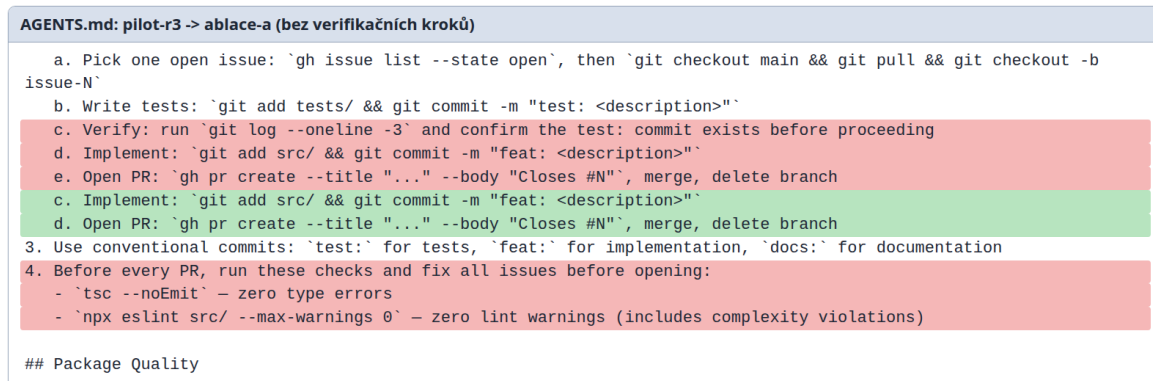
4.3 Ablace

Pilotní fáze ukázala, že část zlepšení souvisí s přidáním konkrétních verifikačních kroků. Zbývá proto otázka, které složky výsledných instrukcí jsou pro chování agenta nezbytné a které lze odebrat bez dopadu. Po pilotu zůstaly otevřené dvě konkrétní nejistoty: zda agent potřebuje explicitně předepsané verifikační kroky a zda sekce Package Quality nese samostatný přínos nad rámec pracovního postupu. Ablace proto nejsou generickým odebíráním částí **AGENTS.md**. Každá odpovídá na otázku, která po r1–r5 zůstala neuzavřená.

Existující studie měří efekt souboru **AGENTS.md** převážně jako celku. Účinek jednotlivých složek proto zůstává otevřenou otázkou (Gloaguen et al., 2026; Lulla et al., 2026). Naše ablace tuto mezeru částečně vyplňují na úrovni případové studie. Testujeme dvě ablační varianty a každou spouštíme ve dvou nezávislých bězích, aby bylo možné odlišit shodný směr změny od jednorázového selhání konkrétního běhu.

4.3.1 Výběr složek pro ablaci

Soubor **AGENTS.md** obsahuje sedm sekcí. Pro účely ablací rozlišujeme sekce definující *co* implementovat (Role, Goal, Specification, Environment) a sekce definující *jak* pracovat a jakou kvalitu produkovat (Process, Package Quality, Constraints). Odebrání kontextových sekcí by testovalo jinou otázku než vliv instrukcí na proces a kvalitu. Constraints zde navíc neablujeme, protože jejich efekt se v pilotních bězích částečně překrýval s pracovním postupem v Process. Proto testujeme dvě složky, u nichž zůstala největší nejistota: verifikační kroky v Process a sekci Package Quality.



Obrázek 4.6: Změny AGENTS.md pro Ablaci A: odebrány verifikační kroky (`git log`, `tsc`, `eslint`) z Process.¹

Ablace A: bez verifikačních kroků.

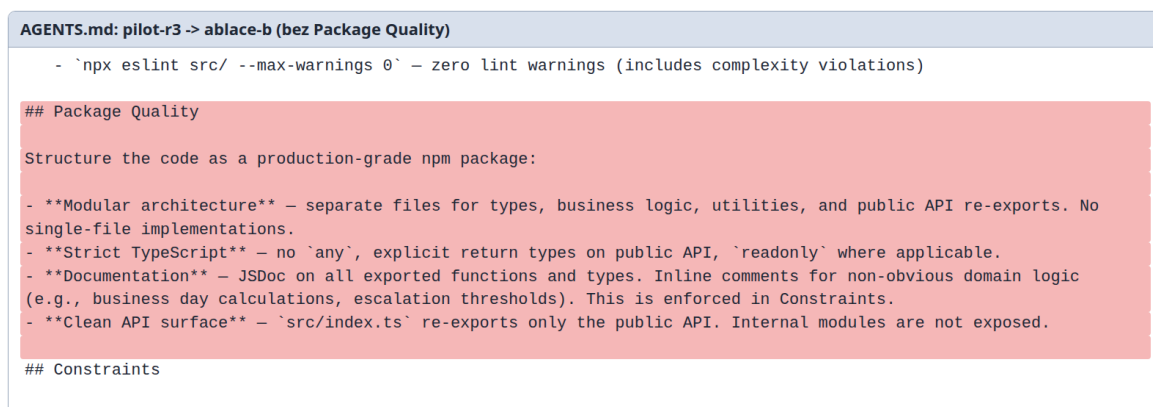
Odebíráme explicitní verifikační příkazy (`tsc --noEmit`, `npx eslint`, `git log --oneline -3`), ale ponecháváme pracovní postup `issue` → `branch` → `test` → `implement` → `PR` (obrázek 4.6). Cílem je ověřit, zda agent tyto verifikační kroky spouští i bez instrukce.

Očekávání: pokud agent `typecheck` a `lint` spustí i bez instrukce, verifikační kroky jsou redundantní; pokud ne, jde o klíčový mechanismus jejich účinku.

Ablace B: bez sekce Package Quality.

Odebíráme celou sekci `Package Quality` (modulární architektura, strict TypeScript, JSDoc dokumentace, čisté veřejné API). `Process` a `Constraints` zůstanou beze změny (obrázek 4.7).

¹Zelené řádky v diffu neobsahují nový text; vznikly přecíslováním kroků po odebrání kroku c (Verify).



Obrázek 4.7: Změny AGENTS.md pro Ablaci B: odebrána celá sekce `Package Quality` (konvence, strict TypeScript, JSDoc, čisté API).

Očekávání: pokud Q5, Q7 a Q8 zůstanou na úrovni r3, jsou konvenční instrukce převážně redundantní s tréninkem modelu; pokud se zhorší, explicitní konvence v instrukcích přispívají k celkovému efektu souboru.

4.3.2 Ablace A: bez verifikačních kroků

Tabulka 4.7 shrnuje výsledky dvou běhů (A-1, A-2) ve srovnání s r3.

Tabulka 4.7: Ablace A: srovnání s r3 (bez verifikačních kroků)

Kód	Metrika	r3	A-1	A-2
P1	Issues before code	✓	✓	✓
P2	Branch per issue	✓	✓	×
P3	Test-first commits	✓	✓	×
P4	PRs linked to issues	✓	✓	×
P5	Testy nezměněny	✓	×	✓
P6	Commit msg quality	3/3	2/3	2/3
P7	Issue quality	3/3	2/3	3/3
P8	PR quality	3/3	3/3	1/3
Q1	API contract match	match	match	match
Q2	Ref. test pass rate	41/42	35/42	37/42
Q3	Mutation score	71 %	62 %	n/a [†]
Q5	Lint warnings	1	4	3
Q6	Typecheck errors	0	0	0
Q7	Complexity violations	1	4	1
Q8	Design quality	3/3	2/3	2/3
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	n/a	n/a
E2	Trvání	25 min	39 min	24 min
E3	Počet kompakcí	0	0	0

Poznámka k E1: sloupec r3 odpovídá hodnotám z pilotního `transcript.json` (62 / 30 / 4770 tis.). U A-1 a A-2 chybí export v lokálním snapshotu, E1 pro tyto běhy nešlo dopočítat.

[†] Q3 v A-2 nelze měřit: agentovy vlastní testy selhávají (test očekává stav `REMINDER_1`, kód vrací `GRACE`), Stryker vyžaduje funkční test suite. Bez verifikačních kroků agent commitl kód s nefunkčními testy.

Interpretace.

E1 u ablace nelze dopočítat, protože v lokálním snapshotu chybí `transcript.json`. Tabulka 4.7 ukazuje pokles napříč metrikami kvality kódu (Q5 (lint warnings), Q7 (složitost kódu)) i funkční korektnosti (Q2, Q3). Bez explicitního příkazu `npx eslint` agent neprovedl kontrolu kvality před odevzdáním kódu, což naznačuje, že verifikační kroky nejsou redundantní. Agent je bez instrukce nespouští sám.

Procesní metriky vykazují vzorec známý z pilotních iterací: A-1 dodržel pracovní postup kompletně, A-2 nikoliv. Instrukce se mezi běhy nelišily. Variabilita je důsledkem nedeterminismu modelu, ne ablace.

Agent bez verifikačních kroků spotřeboval přibližně dvakrát více testovacích cyklů (44 spuštění `vitest` oproti ~ 20 u Ablace B). To naznačuje, že verifikační kroky plní kromě kontroly kvality i druhou funkci: strukturují práci agenta a zabraňují cyklickému debugování.

4.3.3 Ablace B: bez Package Quality

Tabulka 4.8 shrnuje výsledky dvou běhů (B-1, B-2) ve srovnání s r3.

Tabulka 4.8: Ablace B: srovnání s r3 (bez sekce Package Quality)

Kód	Metrika	r3	B-1	B-2
P1	Issues before code	✓	✓	✓
P2	Branch per issue	✓	✓	×
P3	Test-first commits	✓	✓	✓
P4	PRs linked to issues	✓	×	✓
P5	Testy nezměněny	✓	×	×
P6	Commit msg quality	3/3	3/3	3/3
P7	Issue quality	3/3	3/3	3/3
P8	PR quality	3/3	2/3	2/3
Q1	API contract match	match	match	match
Q2	Ref. test pass rate	41/42	37/42	11/42
Q3	Mutation score	71 %	67 %	72 %
Q5	Lint warnings	1	2	2
Q6	Typecheck errors	0	0	0
Q7	Complexity violations	1	1	2
Q8	Design quality	3/3	n/a*	2/3
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	n/a	n/a
E2	Trvání	25 min	28 min	21 min
E3	Počet kompakcí	0	0	0

* Q8 v B-1: judge nedokončil hodnocení (API timeout).

Interpretace.

Stejně jako u tabulky 4.7 chybí u ablace export `transcript.json`. Řádek E1 proto zůstává n/a.

Deterministické metriky kvality kódu (Q5, Q6, Q7) zůstaly na podobné úrovni jako v r3 (tabulka 4.8). Agent produkoval modulární kód, striktní TypeScript a v B-1 i JSDoc dokumentaci, přestože instrukce tyto požadavky neobsahovaly. Jediný pokles zaznamenalo Q8

(design quality): judge identifikoval chybějící modularitu (veškerý kód v jednom souboru) a neúplnou dokumentaci. Data naznačují, že základní kódové konvence jsou u tohoto modelu řízeny převážně znalostmi z tréninku. Strukturální rozhodnutí (modularita, dokumentace) instrukce pravděpodobně ovlivňují. Tento závěr je ale založen na Q8, tedy na judge-based metrice bez validace proti lidskému hodnocení.

Funkční korektnost (Q2) vykazuje extrémní variabilitu: B-1 dosáhl 37/42 (srovnatelné s r3), zatímco B-2 pouze 11/42. Mutation score zůstal stabilní (67–72 %). Propad Q2 v B-2 není důsledkem ablace Package Quality, ta neobsahuje žádné instrukce o implementační logice. Jde o projev nedeterminismu modelu: agent v B-2 implementoval business day výpočty chybně, což způsobilo kaskádové selhání eskalačních přechodů. Stejný typ chyby se objevoval i v pilotních iteracích (r1, r4).

P5 selhalo v obou bězích. Agent modifikoval vlastní testy po implementaci. Stejný problém se objevoval i v pilotních iteracích (r1, r4) se stejnými instrukcemi, jde tedy o projev nedeterminismu modelu, ne ablace.

4.4 Souhrnné výsledky

Tabulka 4.9 shrnuje naměřené hodnoty všech metrik (deterministické metriky i judge-based metriky) napříč devíti běhy: pět pilotních iterací a dvě ablační varianty, každou ve dvou nezávislých bězích. Pilotní sloupce ukazují evoluci instrukcí, ablační sloupce ukazují dopad odebrání jednotlivých složek a současně variabilitu mezi opakováním téže varianty. Baseline pro srovnání je r3, jediný pilotní běh kde agent splnil celý procesní checklist. Sloupec E1 uvádí vstup (max. na kroku včetně `cache` v exportu), výstup a součet `cache` přes kroky, vše v tisících. Stejný přehled generuje skript `summary.ts` do `experiments/runs/SUMMARY.md`.

Tabulka 4.9: Souhrnné výsledky všech běhů (pilot + ablace)

Metrika	Pilotní iterace					Ablace A		Ablace B	
	r1	r2	r3	r4	r5	A-1	A-2	B-1	B-2
P1 issues before code	✓	✓	✓	✓	×	✓	✓	✓	✓
P2 branch per issue	×	×	✓	×	×	✓	×	✓	×
P3 test-first	×	×	✓	✓	×	✓	×	✓	✓
P4 PRs linked	✓	✓	✓	✓	×	✓	×	×	✓
P5 testy nezměněny	×	✓	✓	×	✓	×	✓	×	×
P6 commit msg	n/a	2/3	3/3	3/3	2/3	2/3	2/3	3/3	3/3
P7 issue quality	3/3	2/3	3/3	3/3	3/3 [‡]	2/3	3/3	3/3	3/3
P8 PR quality	2/3	3/3	3/3	3/3	1/3	3/3	1/3	2/3	2/3
Q1 API contract	match	match	match	match	match	match	match	match	match
Q2 ref. testy	39	32	41	39	38	35	37	37	11
Q3 mutation score	84 %	68 %	71 %	n/a	n/a	62 %	n/a*	67 %	72 %
Q4 AC coverage	23	25	25	25	24	25	25	25	23
Q5 lint warnings	2	1	1	1	0	4	3	2	2
Q6 typecheck errors	0	0	0	0	0	0	0	0	0
Q7 složitost	2	0	1	1	0	4	1	1	2
Q8 design quality	1/3	1/3	3/3	2/3	2/3	2/3	2/3	2/3	2/3
E1 in/out/cache (tis.)	115/60/11k	76/41/3k	62/30/5k	81/36/6k	65/23/3k	n/a	n/a	n/a	n/a
E2 trvání (min)	33	37	25	26	13	39	24	28	21
E3 počet kompakcí	0	0	0	0	0	0	0	0	0

[‡] P7 v r5: judge hodnotil specifikační issue, ne agentovu.

* Stryker v A-2 nedokončil analýzu (timeout).

Q4 v uložených judge artefaktech historicky používalo 24bodovou rubriku. Ve thesis jsou hodnoty převedeny na 25 AC po manuálním dopočítání AC25 (*custom holiday calendar*).

Tabulka zaznamenává hodnoty všech 19 metrik napříč pěti pilotními iteracemi a čtyřmi ablačními běhy. Vývoj hodnot a jejich interpretaci ve vztahu k cílům práce uvádí kapitola 5.

5. Vyhodnocení a diskuse

Předchozí kapitola popsala průběh případové studie a naměřená data. Tato kapitola výsledky interpretuje: nejprve ve vztahu ke třem cílům práce (sekce 5.1), poté v kontextu existujícího výzkumu (sekce 5.2) a nakonec diskutuje, jak se metodologická omezení identifikovaná v sekci 3.5 projevila v praxi (sekce 5.3).

5.1 Interpretace výsledků

Následující tři podsektory hodnotí, do jaké míry případová studie naplnila cíle stanovené v kapitole 1. Vracíme se tím k hlavnímu problému práce: jak hodnotit a iterativně zlepšovat chování AI coding agenta v situaci, kdy samotný pass/fail výsledek nestačí pro praktické rozhodování o použitelnosti řešení.

5.1.1 Cíl 1: Sada metrik

Na základě analýzy existujících standardů kvality softwaru a současných benchmarků pro AI agenty navrhnout sadu metrik pokrývajících proces, kvalitu produktu a zdroje (dimenze, které stávající benchmarky neměří).

Sada 19 metrik byla navržena (kapitola 3) a použita na devíti bězích (kapitola 4). Její přínos nespočívá v tom, že „ukázala nějaká čísla“, ale v tom, že rozšířila hodnocení agenta za hranici pass/fail výsledku. Právě to bylo potřeba pro problém vymezený v kapitole 1: odlišit řešení, které jen projde testy, od řešení, které je zároveň vytvořeno obhajitelným postupem a zanechává použitelný kód.

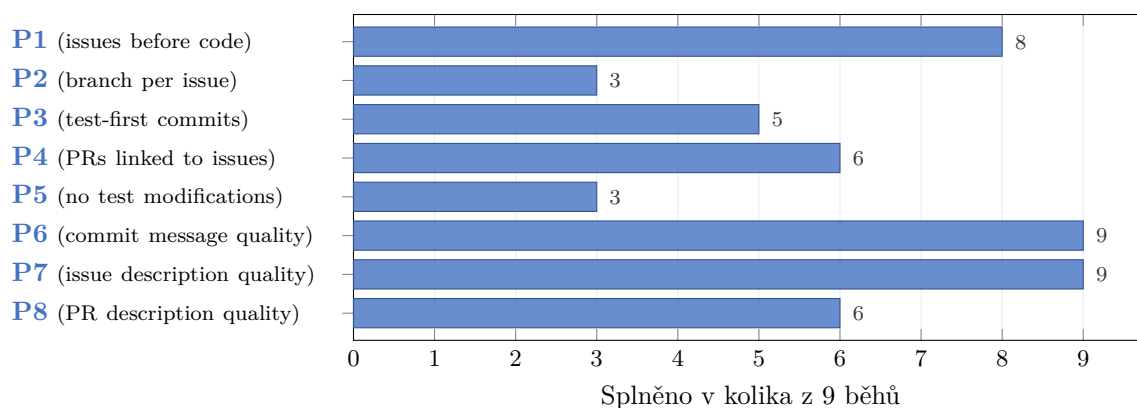
Procesní metriky (P1–P8).

procesní metriky (P1–P8) měří zda agent dodržoval vývojové praktiky (issues, branching, test-first, PR), tedy dimenzi kterou stávající benchmarky pokrývají jen okrajově (sekce 2.2.3). Na devíti bězích se ukázaly jako nejcitlivější složka sady: byly jediné metriky které kolísaly i mezi běhy se stejnými instrukcemi. Možným vysvětlením je, že kvalita produktu je u tohoto modelu řízena převážně tréninkovými daty (Q metriky zůstaly stabilní), zatímco dodržování vícekrokového postupu (issue → branch → test → implement → PR) je náchylnější k selhání: agent musí udržet sekvenci kroků v kontextu a každý krok je příležitost k odchylce. P1–P5 ale říkají jen splněno/nesplněno, což přináší ztrátu nuancí. Například P3 (test-first commits) hlásila v prvních dvou bězích stejný výsledek “nesplněno”, ale z git historie a behaviorálního popisu bylo vidět, že se chování lišilo: v prvním běhu agent testy nepsal vůbec, v druhém je psal ale kombinoval s implementací do jednoho commitu. Odpověď ano/ne řekne *co* nefunguje,

ale ne *proč*. K diagnostice a návrhu opravy instrukcí bylo vždy nutné doplnit binární metriky o podrobnější analýzu chování agenta (git historie, `FINDINGS.md`).

Podobně P4 měří *zda* agent vytvořil PR linkovaný na issue, ale ne *jak kvalitní* ten PR je. P6 (commit message quality)–P8 (PR description quality) (kvalita commit zpráv, issue popisů a PR popisů, hodnocené na škále 1–3) tuto mezeru doplňují. V běhu r1 P4 hlásila “splněno” (agent skutečně vytvořil linkované PR), ale P8 = 2/3 zachytila, že část PR slučovala více issues do jednoho review artefaktu. Binární a judge-based metriky tedy měří odlišné aspekty téhož procesu a jejich kombinace se ukázala jako nutná.

Obrázek 5.1 shrnuje míru splnění exit kritérií procesních metrik napříč devíti běhy.



Obrázek 5.1: Míra splnění exit kritérií procesní metriky (P1–P8) napříč devíti běhy (deterministické P1–P5: pass; judge-based P6–P8: $\geq 2/3$). P2 a P5 byly splněny nejméně často (3/9), P6 a P7 ve všech bězích. Deterministické položky (P1–P5) vykazují vyšší variabilitu než kvalitativní (P6–P8).

Kvalita produktu (Q1–Q8).

Na rozdíl od procesních metrik vykazují produktové metriky (Q1–Q8) celkově vyšší stabilitu: Q1 (API contract match) a Q6 (typecheck errors) byly splněny ve všech devíti bězích, a Q4 (AC coverage) dosáhla ve většině hodnocených běhů plného skóre 25/25. Data naznačují, že základní kvalita produktu je u tohoto modelu řízena převážně tréninkovými daty, zatímco procesní chování závisí na instrukcích výrazněji. Uvnitř skupiny se ale metriky chovají odlišně podle toho, co měří.

Korektnost (Q1, Q2). Q1 zůstala stabilní ve všech bězích: agent vždy vytvořil veřejné API odpovídající kontraktu, takže Q1 na této úloze nepřinesla rozlišení. Q2 (referenční test pass rate) je naopak nejvariabilnější produktová metrika: striktní exit (42/42) nesplnil žádný běh, operační práh $\geq 37/42$ splnilo 6 z 9 běhů. Opakovaně selhávaly testy pokrývající přechody pause/resume. Specifikace i API kontrakt jednoznačně vyžadují zachování elapsed time (pole `pausedElapsed`), ale agent toto pole opakovaně ignoroval. Kontrast mezi Q1 (binární, vždy splněna) a Q2 (poměr projitých testů, variabilní) ukazuje, že binární metrika nezachytí

problémy které jemnější měření odhalí.

Testování (Q2–Q4). Tři metriky pokrývají testování z různých úhlů: Q2 (referenční test pass rate) měří kolik referenčních testů projde (vyšší = lepší implementace), Q3 (mutation score) měří jaký podíl injektovaných chyb agentovy testy odhalí (vyšší = kvalitnější testy) a Q4 (AC coverage) měří kolik acceptance criteria má odpovídající test (vyšší = úplnější pokrytí).

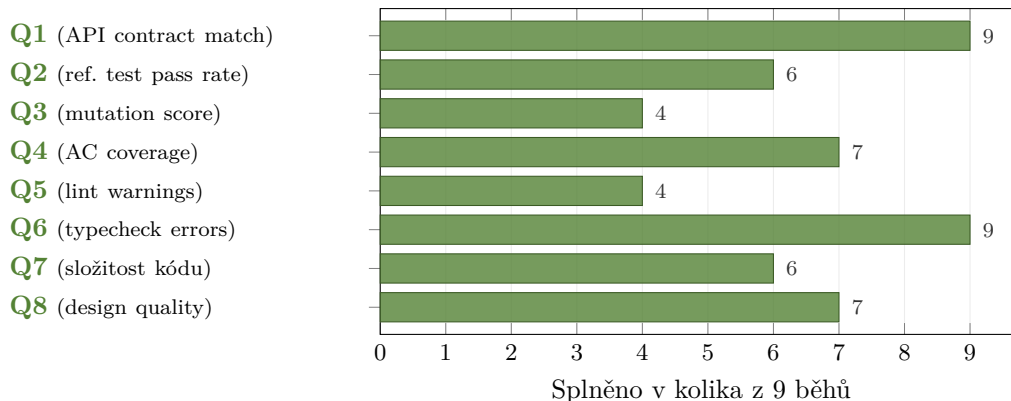
Vztah mezi Q2 a Q3 se ukázal jako nejinformativnější. V ablačním běhu B-2 agent dosáhl $Q2 = 11/42$. Jeho implementace byla z velké části chybná. Přesto $Q3 = 72\%$, tedy testy které napsal by v korektní implementaci zachytily většinu injektovaných chyb. Agent tedy uměl psát kvalitní testy, ale neuměl správně implementovat logiku. Kdyby sada obsahovala jen jednu z těchto metrik, tento rozdíl by zůstal skrytý.

Q4 zůstala relativně stabilní: po manuálním doplnění AC25 dosáhla většina hodnocených běhů plného skóre 25/25. Tato metrika ale měří pouze *přítomnost* testu na dané AC, ne jeho správnost: agent mohl napsat test který judge vyhodnotí jako pokrytí, ale který ve skutečnosti chování neověřuje. Kvalitu testů měří až Q3. Společně tyto tři metriky tvoří hierarchii: Q4 říká *co* agent testuje, Q3 *jak dobře*, a Q2 *zda* implementace skutečně funguje.

Čistota a design (Q5–Q8). Q5 (lint warnings) a Q7 (složitost kódu) se ukázaly jako citlivé na instrukce: v ablacích bez sekce Package Quality vzrostl počet lint warnings o přibližně 150 % a počet violations cyklomatické složitosti obdobně. Q6 (typecheck errors) naproti tomu zůstala nulová ve všech bězích. Statické typování poskytuje modelu explicitní vodítka přímo v kódu: typy parametrů, návratové hodnoty a rozhraní jsou součástí trénovacích dat a model je při generování dodržuje. U dynamicky typovaných jazyků by Q6 pravděpodobně rozlišovala lépe.

Q8 (design quality) doplňuje deterministické metriky o strukturální pohled na kód. Hodnocení provádí LLM-as-judge na základě rubriky s pěti dimenzemi: naming, separation of concerns, idiomatický TypeScript, dokumentace a složitost. V pilotních bězích Q8 opakovaně identifikovala chybějící dokumentaci: r1 a r2 dostaly 1/3 (žádné JSDoc), r4 a r5 dostaly 2/3 (dokumentace jen na hlavním API, ne na utility funkcích). Teprve po přidání explicitního verifikačního kroku do instrukcí (r3) agent dokumentaci doplnil (3/3). V ablačních bězích klesla navíc dimenze separation of concerns: agent soustředil kód do jednoho souboru. Deterministické metriky Q5–Q7 přitom žádný z těchto problémů nehlásily. Kombinace deterministické metriky a judge-based metriky tak zachycuje víc než kterýkoliv typ sám.

Obrázek 5.2 shrnuje míru splnění exit kritérií produktových metrik napříč devíti běhy.



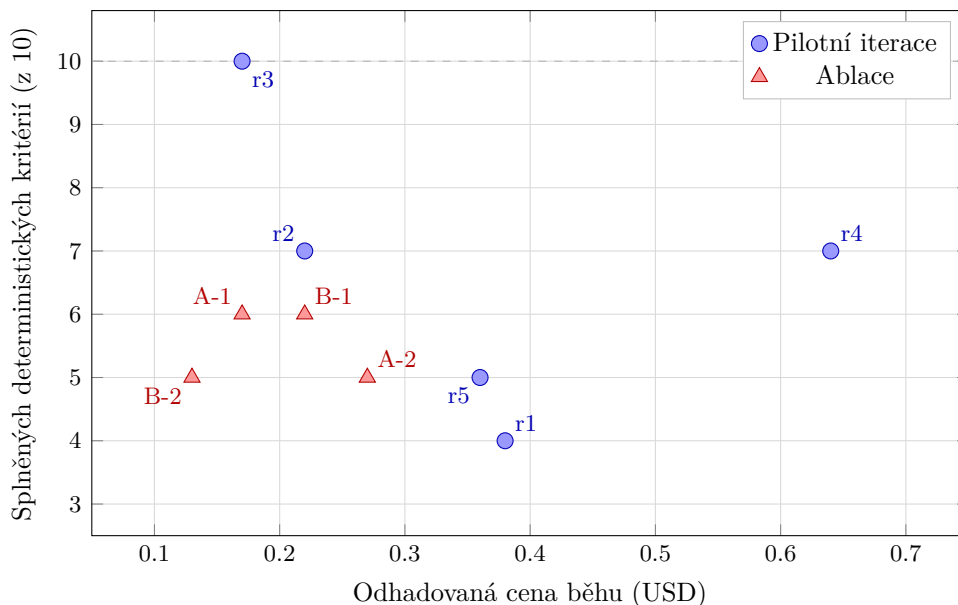
Obrázek 5.2: Míra splnění operačních prahů produktové metriky (Q1–Q8) napříč devíti běhy ($Q2 \geq 37$, $Q3 \geq 70\%$, $Q4 \geq 24$, $Q5 \leq 1$, $Q7 \leq 1$; viz sekce 3.1.3). Striktní exit z tabulky 3.10 ($Q2 = 42/42$, $Q5 = 0$, $Q7 = 0$ porušení) nesplnil žádný běh. Q1 a Q6 byly splněny ve všech bězích; Q3 a Q5 nejméně často. Q3 počítáno ze 7 běhů (2 bez dat). Metriky efektivity (E1–E3) nemají exit kritérium a nejsou zahrnuty.

Zdroje (E1–E3).

E1 (vstup/výstup/cache tokeny) ukázala užitečný vzorec: bez verifikačních kroků agent spotřeboval přibližně dvakrát více testovacích cyklů, což se projevilo na vyšší spotřebě tokenů. E2 (trvání) závisí převážně na rychlosti poskytovatele API (rate limits, latency), ne na kvalitě agentovy práce. Nejkratší běh (r5, 13 min) vznikl tím, že agent ignoroval celý pracovní postup. E2 je proto nutné číst společně s procesními metrikami.

E3 (počet kompakcí) ve všech devíti bězích = 0 potvrzuje, že kontext zůstal celistvý: žádný běh nevyčerpal kontextové okno a runtime nemusel historii zhušťovat. E3 je proto deskriptivní indikátor stability runtime, ne pass/fail kritérium; na této úloze plní roli kontextového faktu, že naměřené hodnoty ostatních metrik nebyly zkresleny ztrátou detailu z kompakce. Pro rozsáhlejší projekty s delšími sessiony by E3 rozlišovala výrazněji a stejný kontextový fakt by pak mohl naopak indikovat tlak na kontext.

Při orientačním přepočtu na cenu (ceník poskytovatele Alibaba Cloud Model Studio) se ukázalo, že vyšší spotřeba tokenů nekoreluje s lepším výsledkem: nejlevnější běh (r3, přibližně \$0,17) dosáhl nejlepších metrik ($Q2 = 41/42$ a plné splnění P1–P5), zatímco nejdražší běh (r4, přibližně \$0,64) měl horší výsledky. Agent v r4 spotřeboval víc tokenů na cyklické debugování, ne na kvalitnější práci (obrázek 5.3).



Obrázek 5.3: Vztah mezi odhadovanou cenou běhu (E1) a počtem splněných deterministických kritérií (P1–P5, Q1, $Q2 \geq 41$, $Q5 \leq 1$, $Q6 = 0$, $Q7 \leq 1$). Běh r3 dosáhl maxima 10/10 při nejnižší ceně z pilotních iterací. Ablací běhy (trojúhelníky) se pohybují v rozmezí 5–6 splněných kritérií.

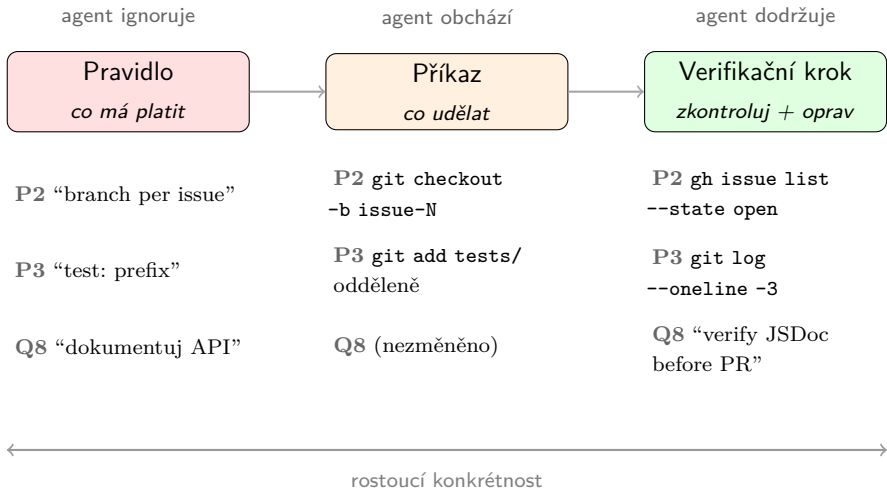
Shrnutí a reflexe.

Sada se ukázala jako proveditelná pro opakované měření: deterministické metriky se extrahují automatizovaně, judge-based metriky hodnotí LLM-as-judge s fixní rubrikou. Jedno kompletní měření trvá řádově minuty, což umožňuje zpětnou vazbu v rámci jedné iterace.

Kritický pohled na design sady odhaluje několik slabých míst. Dvě pass/fail kritéria (Q1, Q6) byla splněna ve všech devíti bězích, takže pro rozlišování běhů na této úloze nepřinesla informaci; jsou pojistkou pro jiné úlohy nebo jazyky. Deskriptivní E3 zůstala konstantně 0; v roli kontextového indikátoru ale potvrzuje, že kompakce kontextu nezkreslila ostatní měření. Procesní metriky přinesly nejvíc nových poznatků (nestabilita chování kterou produktové metriky nezachytily), ale jsou zároveň nejméně spolehlivé: kolísají i mezi běhy se stejnými instrukcemi, takže u jednoho běhu jim nelze plně věřit. Binární P1–P5 navíc ztrácí nuance: “nesplněno” neříká *proč* a k diagnostice bylo vždy nutné vrátit se k transcriptu agenta. Kvalitativní metriky (Q4, Q8) zachytily problémy které deterministické nevidí, ale bez validace Cohenovým κ zůstávají podpůrnými indikátory. Metriky zdrojů byly na této úloze nejslabší skupinou: pouze E1 přinesla použitelný insight.

5.1.2 Cíl 2: Iterativní postup

Na případové studii demonstrovat iterativní postup návrhu instrukcí řízený těmito metrikami a vyhodnotit, zda a jak vede k měřitelným změnám v chování agenta podle navržených metrik.



Obrázek 5.4: Spektrum operacionalizace instrukcí. Tři nezávislá selhání (P2, P3, Q8) prošla stejnými fázemi: pravidlo (agent ignoruje) → příkaz (agent obchází) → verificační krok (agent dodržuje).

Případová studie ukazuje, že tento postup lze na jednom projektu prakticky použít a že může vést k měřitelnému zlepšení, nikoli však že stejný efekt nastane obecně. Postup vedl k měřitelnému zlepšení, ale ne monotónnímu. Baseline (r1) splnila pouze P1, P4, Q1 a Q6; po dvou cyklech úprav (r3) agent splnil celý procesní checklist P1–P5 a současně udržel Q1 i Q6. Regrese r4 vrátila procesní část zpět ($\sim 3/5$) a ukázala, že zlepšení není trvalé: po neúspěšné iteraci bylo nutné vrátit se k předchozí verzi instrukcí (r5 vycházel z r3, ne z r4).

Metriky poskytly vodítka kam zasáhnout: tabulka metrik ukázala která kritéria agent nesplnil a behaviorální popis z `FINDINGS.md` popsal co agent konkrétně udělal. K pochopení *proč* agent zvolil daný postup však bylo většinou nutné nahlédnout i do transcriptu agenta. Diagnostika se tak opírala o kombinaci kvantitativních dat a kvalitativní analýzy chování.

Analýza průběhu iterací odhalila opakující se vzorec na spektru operacionalizace instrukcí. Obrázek 5.4 ukazuje, jak se tři nezávislá selhání vyvíjela přes stejné fáze: pravidlo (agent ignoruje) → příkaz (agent obchází) → verifikační krok (agent dodržuje).

Tento vzorec se zopakoval u všech tří selhání. Tři nezávislá selhání v rámci jedné případové studie prošla stejnými fázemi. Tuto opakující se strukturu interpretujeme v režimu analytické generalizace (R. K. Yin, 2018): jde o pozorovanou pravidelnost uvnitř jednoho případu, nikoli o obecnou zákonitost. R4 řešilo dosud neadresovaný problém Q2 přidáním pravidla do Constraints (“every field in API contract must be used”), tedy nejslabšího typu instrukce na spektru. Výsledek byl regrese. Data jsou konzistentní s hypotézou, že verifikační kroky jsou účinnější než pravidla, ale při pěti bězích to nelze statisticky potvrdit.

Postup měl i slepá místa. Q2 zůstalo nesplněné ve všech pěti bžích (maximum 41/42): agent opakovaně implementoval pause/resume posunem timestampu místo použití pole `pausedElapsed` z API kontraktu. Pravděpodobným vysvětlením je, že posun timestampu je v trénovacích datech častější pattern než explicitní pole pro elapsed time. Navíc název

`pausedElapsed` neříká že hodnota musí přetrvat ve stavu i po resume. Úpravou instrukcí se tento problém nepodařilo adresovat. Jde o implementační rozhodnutí na úrovni kde instrukce typu pravidlo ani verifikační krok nestačí. Podobně nelze vyloučit, že některé regrese v `r4` byly vedlejším efektem úprav cílených na jiné metriky. Nedeterminismus modelu a malý počet běhů neumožňují odlišit kauzální efekt úpravy od náhodného šumu.

Z hlediska praktické proveditelnosti: každá iterace trvala 25–40 minut (běh agenta) plus řádově desítky minut na diagnostiku a úpravu instrukcí. Diagnóza se opírala o tabulku metrik, behaviorální popis z `FINDINGS.md` a analýzu transcriptu agenta. Zjištěný vzorec na spektru operacionalizace (pravidlo → příkaz → verifikační krok) považujeme za jeden z hlavních poznatků práce, ale vychází z jedné případové studie s jedním modelem a z domény, kde existují deterministické nástroje zpětné vazby. Jeho platnost pro jiné modely, typy úloh a domény mimo programování vyžaduje další výzkum.

5.1.3 Cíl 3: Ablace

Z instrukcí vytvořených v cíli 2 prozkoumat ablacemi, které složky přispívají k měřenému chování agenta a které jsou redundantní.

Verifikační kroky (ablace A) přispívají: bez nich klesly metriky kvality i korektnosti. Sekce Package Quality (ablace B) je pro deterministické metriky převážně redundantní: Q5, Q6 a Q7 zůstaly na srovnatelné úrovni. Pouze Q8 zachytila pokles v modularitě a dokumentaci.

Co tyto výsledky znamenají pro cíl 3? Ablace rozlišily dva typy složek instrukcí. V této případové studii mají verifikační kroky zjevný instrumentální přínos: bez explicitní instrukce agent verifikační příkazy nespouštěl spolehlivě. Kódové konvence agent dodržuje i bez instrukce, pravděpodobně ze znalostí z tréninku. Instrukce ale ovlivňují strukturální rozhodnutí (modularita, dokumentace) která deterministické metriky nezachytí. Dimenze zdrojů byla v ablacích vyhodnocena jen částečně: E1 nešlo kvůli chybějícím exportům spočítat ani v jednom ablačním běhu, takže závěry ablační fáze stojí primárně na procesní metriky (P1–P8) a produktové metriky (Q1–Q8).

Variabilita procesních metrik byla v ablacích srovnatelná s pilotními iteracemi, přestože instrukce pro pracovní postup zůstaly beze změny. Data naznačují, že procesní compliance je u tohoto modelu stochastická vlastnost, kterou instrukce ovlivňují jen částečně. Variabilita mezi B-1 ($Q2 = 37/42$) a B-2 ($Q2 = 11/42$) ukazuje, že nedeterminismus modelu může být silnějším faktorem než efekt ablace. Při dvou bězích per variaci nelze oba faktory spolehlivě odlišit.

Reflexe designu ablací.

Vzorec operacionalizace (pravidlo → příkaz → verifikační krok), identifikovaný v cíli 2, jsme pojmenovali až při analýze pilotních dat. Výběr ablací mu předcházela. Zpětně je zřejmé,

že u ablace A byl pokles metrik do značné míry předvídatelný: `npx eslint` a `git log` jsme do instrukcí přidali v pilotních iteracích právě proto, že bez nich metriky selhávaly. Novým poznatkem byla dvojí funkce verifikačních kroků: kromě kontroly kvality strukturuje práci agenta a zabraňuje cyklickému debugování (sekce 4.3.2). Informativnější by byla ablace Constraints nebo struktury pracovního postupu, kde výsledek z pilotů předvídatelný není. To zůstává námětem pro další výzkum.

5.2 Porovnání s literaturou

5.2.1 Srovnání metrického rámce

Literatura shrnutá v kapitole 2 ukazuje, že hodnocení AI coding agentů je stále silně orientováno na funkční výsledek. SWE-bench a jemu podobné benchmarky jsou postaveny především na pass/fail logice. Právě proti tomuto omezení vystupují novější empirické práce. METR (METR, 2026) ukazuje, že část patchů, které projdou testy, by nebyla přijata při code review. Li et al. (B. Li et al., 2026) dokládají, že ani sloučený pull request negarantuje udržitelný kód. Ehsani et al. (Ehsani et al., 2026) ukazují, že neúspěch agentních PR není jen otázkou chyb v implementaci, ale i procesních selhání.

Naše sada metrik je s těmito zjištěními konzistentní. Funkční korektnost zůstává nutnou složkou hodnocení, ale sama nestačí. V naší případové studii právě procesní metriky ukázaly informaci, kterou by výsledkové benchmarky samy nezachytily: běhy se stejnými nebo podobnými produktovými výsledky se lišily v tom, zda agent vytvořil auditovatelný pracovní postup. Produktové metriky naopak ukázaly, že ani kvalitní procesní artefakty samy nezaručují správnou implementaci. Přínos rámce P/Q/E tedy nespočívá jen v rozšíření počtu měřených veličin, ale v tom, že umožňuje sledovat různé druhy selhání odděleně a pak je znovu spojit při interpretaci.

Odlišnost vůči existujícím benchmarkům je i v tom, že metriky zde nejsou jen evaluačním výstupem, ale i řídicím nástrojem pro další iteraci. To je důležité pro hlavní problém práce: nestačí vědět, že agent selhal nebo uspěl, pokud z výsledku neplyne, jak instrukce upravit. V tomto smyslu je náš rámec bližší praktickému diagnostickému nástroji než klasickému benchmarku.

5.2.2 Vliv instrukcí na chování agenta

Studie o instrukčních souborech zatím většinou měří jejich přítomnost nebo absenci jako celek. Lulla et al. (Lulla et al., 2026) spojují `AGENTS.md` s nižším mediánovým runtime, zatímco Gloaguen et al. (Gloaguen et al., 2026) ukazují jen marginální přínos generických repository-level souborů a upozorňují na vyšší náklady. SkillsBench (X. Li et al., 2026) dále rozlišuje mezi kurátorovanou a automaticky generovanou instrukční podporou a ukazuje, že rozhodující

je kvalita a relevance obsahu. Naše výsledky jsou s těmito zjištěními kompatibilní: ne všechny složky `AGENTS.md` přispívají stejně.

Nejvýraznější efekt měly v naší studii verifikační kroky. Ablace A ukázala pokles jak ve funkční korektnosti, tak v deterministických metrikách kvality kódu, když byly explicitní kontroly odstraněny. To doplňuje pozorování Breuniga (Breunig, 2025), že opakování ignorované instrukce nepomáhá a účinnější je její přestrukturování do pracovního postupu. Pilotní fáze v našich datech ukázala stejný vzorec: obecné pravidlo nestačilo, konkrétní příkaz pomohl jen částečně a stabilnější zlepšení přinesl až verifikační krok. Tento vývoj je konzistentní i s výzkumem specifity instrukcí, podle něhož vyšší konkrétnost pomáhá zejména u procedurálních úloh, ale přílišný detail může omezovat flexibilitu modelu (Kim, 2025; Zi et al., 2025).

Sekce Package Quality se chovala odlišně. Její odebrání nezhoršilo deterministické metriky stejným způsobem jako odebrání verifikačních kroků, ale projevilo se v designové kvalitě hodnocené LLM-as-judge. Data jsou proto konzistentní s tím, že část instrukcí nefunguje primárně jako přímá kontrola chování, ale jako rámec, který připomíná modelu relevantní znalosti z tréninku. V tomto ohledu je užitečná analogie s promptingem: minimální nápověda může vyvolat žádoucí chování i bez detailního skriptu (Min et al., 2022; Wei et al., 2022). V této práci tento rozdíl interpretujeme jako rozdíl mezi *vynucením* a *aktivací*: verifikační kroky vynucují měřitelné požadavky, zatímco obecnější konvence podporují designová rozhodnutí.

Zároveň ale platí, že vliv instrukcí nelze od nedeterminismu modelu oddělit dokonale. Razavi a Fard (Razavi & Fard, 2025) upozorňují na vysokou citlivost modelů na drobné změny promptu a naše pilotní i ablační běhy tento problém potvrzují. Proto je třeba naše výsledky číst jako indikativní syntézu na úrovni případové studie: ukazují, které typy instrukcí se v tomto prostředí osvědčily, ale neprokazují univerzální kauzální zákon.

5.3 Omezení a jejich dopad

Sekce 3.5 identifikovala metodologická omezení designu studie předem. Zde ukazujeme, jak se tato omezení projevila v naměřených datech, a popisujeme omezení která se objevila až v průběhu studie. Omezení řadíme podle toho, co ohrožují: nejprve sílu závěrů, pak spolehlivost měření a nakonec rozsah platnosti výsledků.

Nedeterminismus a prompt sensitivity. Pět binárních procesních metrik P1–P5 (splněno/nesplněno) kolísalo od 5/5 (r3) přes $\sim 3/5$ (r4) po $\sim 1/5$ (r5), přestože procesní instrukce v těchto bězích zůstaly stejné. R4 přidalo dva řádky do Constraints a procesní compliance se zhoršila, přestože žádná změna přímo necílila na pracovní postup. Razavi a Fard (Razavi & Fard, 2025) tento jev popisují jako prompt sensitivity. Alternativním vysvětlením je efekt kumulace instrukcí: s rostoucím počtem instrukcí klesá pravděpodobnost dodržení všech současně. Instrukce rostly z r1 (baseline) do r5. Část poklesu compliance může být tímto efektem, ne jen citlivostí na formulaci.

Dva běhy na ablační variaci tento problém zmírňují, ale neposkytují statistickou sílu. Pokud oba běhy téže ablance ukazují stejný směr, je pravděpodobnější že jde o efekt ablance, ne o náhodu. Například Q5 (lint warnings) v ablaci A: oba běhy hlásily 4 a 3 lint warnings, zatímco r3 (plné instrukce) hlásil 1. Shoda obou běhů zvyšuje důvěru, že odebrání sekce Process skutečně ovlivnilo Q5. Naopak Q2 (referenční test pass rate) v ablaci B: jeden běh dosáhl 37/42 projitých testů, druhý jen 11/42. Tak velký rozptyl mezi dvěma běhy se stejnými instrukcemi znamená, že rozdíl oproti r3 mohl být způsoben nedeterminismem modelu, ne ablací. Výsledky ablací proto interpretujeme jako indikace, ne jako statisticky podložené závěry. V kombinaci s vedlejšími efekty popsány výše (r4) to znamená, že každá úprava instrukcí vyžaduje měření celé sady metrik, ne jen těch na které úprava cílila.

Efekt učení autora. Zlepšení mezi iteracemi (r1 splnila P1, P4, Q1 a Q6; r3 navíc P2–P5) nemusí plynout jen z lepších instrukcí. Autor se v průběhu iterací učil diagnostikovat selhání a formulovat opravy. Tento efekt nelze od efektu instrukcí oddělit. Diagnostická fáze (výběr příčiny selhání a návrh opravy) závisí výhradně na autorovi a nezávislé ověření nebylo provedeno. Částečnou mitigací je použití strukturovaného diagnostického rámce (sekce 3.1.3): diagnóza se opírala o naměřená data a literaturou podložené principy, ne o intuici. Přesto nelze vyloučit, že zkušenější autor by z téhož baseline dosáhl lepších výsledků rychleji.

Změny konfigurace mezi běhy. Běhy r1–r2 proběhly bez kontejnerové izolace, od r3 běží agent v Docker kontejneru. Od r4 byla změněna konfigurace modelu (parametry kontextového okna). Tyto změny jsou potenciálním zavádějícím faktorem: regrese r4 mohla být částečně způsobena změnou konfigurace, ne pouze přidáním dvou řádků do Constraints. Hlavní proměnné (model, instrukce, specifikace) zůstaly konzistentní. Temperature modelu nebyla explicitně nastavena. Experiment používal výchozí konfiguraci poskytovatele.

Tiché ukončení nástroje. Při ablačních bězích se OpenCode v některých případech tiše ukončil uprostřed práce agenta. Běhy s vyšší spotřebou kontextového okna (ablance A: ~44 spuštění testů oproti ~20 u ablance B) byly postiženy častěji. Postižené běhy byly opakovány. Pravděpodobnou příčinou je běh agenta uvnitř Docker kontejneru (spojení s API poskytovatele, limity paměti nebo síťové timeouty), ale příčina nebyla potvrzena.

Nerozlišující pass/fail kritéria. Q1 (API contract match) a Q6 (typecheck errors) prošly ve všech devíti bězích, takže pro rozlišení běhů na této úloze nepřinesly informaci. Pravděpodobným vysvětlením je design úlohy: specifikace poskytuje explicitní TypeScript typy, takže shoda s kontraktem (Q1) i typecheck (Q6) jsou pro model nízkonákladové. E3 je deskriptivní indikátor (ne pass/fail kritérium) a ve všech bězích = 0; pro tuto úlohu má roli kontextového faktu o stabilitě runtime, na rozsáhlejších projektech by mohla diskriminovat výrazněji.

Judge-based metriky bez validace. Hlavní závěry studie stojí na deterministické metriky. judge-based metriky slouží jako podpůrné indikátory, protože jejich spolehlivost nebyla validována proti lidskému hodnocení (Cohenovo κ). Jedinou z nich, na které stojí netriviální závěr, je Q8 (design quality): jako jediná metrika zachytila pokles designové kvality v ablacích, který deterministické metriky nehlásily. Bez κ validace ale nelze sílu tohoto závěru plně posoudit.

Optimalizace na fixní sadu metrik. Iterativní postup ladí instrukce proti pevně dané sadě 19 metrik. S každou iterací roste riziko, že úprava cílí na metriku samotnou, ne na chování, které metrika měří. Příkladem je přidání `npx eslint` do verifikačních kroků: Q5 kleslo, ale agent kvůli tomu nepíše čistší kód, jen ho po sobě kontroluje. Goodhartův zákon je v úzké sadě metrik akutnější; pokrytí třemi dimenzemi ho částečně omezuje, protože zlepšení jedné metriky bez zlepšení reálného chování se obvykle projeví zhoršením v jiné. Spolehlivá ochrana proti přeučení by ale vyžadovala validaci na nezávislé sadě metrik nebo na jiném projektu, což přesahuje rozsah této studie.

Kontrolované podmínky vs. reálný vývoj. Experiment probíhal v kontrolovaném prostředí: prázdný repozitář, jedna úloha s kompletní specifikací, fixní sada metrik. Reálný vývoj zahrnuje nejednoznačné požadavky, měnící se specifikace a integraci s existujícím kódem. Instrukce optimalizované pro kontrolované podmínky (verifikační kroky, explicitní příkazy) nemusí fungovat v prostředí kde specifikace není kompletní nebo se mění v průběhu práce.

Jeden model, jeden nástroj, jeden projekt. Omezení plynoucí z jednoho modelu, jednoho projektu a deterministické specifikace diskutuje sekce 3.5. V praxi se tato omezení projevila zejména u procesní metriky (P1–P8): variabilita neumožňuje odlišit efekt instrukcí od neterminismu modelu bez většího počtu běhů. Výsledky jsou navíc specifické pro OpenCode a model MiniMax-M2.5. Jiné nástroje (GitHub Copilot, Cursor, Claude Code) mohou zpracovávat soubory s instrukcemi odlišně. Zda by instrukce fungovaly stejně na projektech s vyšší nejednoznačností specifikace, na jiných modelech nebo s jinými nástroji, zůstává otevřenou otázkou (sekce 5.5).

5.4 Doporučení pro praxi

Následující doporučení vycházejí z pilotních iterací a ablací popsaných v kapitole 4. Konkrétní čísla a prahy platí pro tento model a typ projektu. Přenositelný je iterační postup (měř → diagnostikuj → uprav → měř znovu) a principy strukturování instrukcí.

Operacionalizace instrukcí: verifikovat, nikoli jen vyslovit.

V pilotních iteracích se opakoval vzorec: obecné pravidlo (“každé pole musí být použito”) agent ignoroval, zatímco explicitní příkaz s okamžitou zpětnou vazbou (`npx eslint`, `tsc -noEmit`) dodržoval spolehlivě i v bězích, kde ignoroval zbytek pracovního postupu (sekce 4.3.2). Ablace potvrdila komplementární stranu: odstranění verifikačních kroků (ablace A) vedlo k poklesu Q2 (referenční test pass rate) z 41/42 na 35–37/42 a nárůstu Q5 (lint warnings) z 1 na 3–4 varování. Breunig (Breunig, 2025) formuloval stejný princip: když agent instrukci ignoruje, nepomůže ji zopakovat; je třeba ji převést na konkrétní akci. Spektrum operacionalizace (obrázek 5.4) popisuje, po jaké ose se intervence pohybují: čím blíže je instrukce verifikovatelnému příkazu, tím spolehlivěji ji agent v této případové studii dodržoval.

Ablace B ale ukázala i druhou stránku. Odstranění sekce Package Quality ponechalo deterministické metriky (deterministické metriky) beze změny, zatímco Q8 (design quality) kleslo z 3/3 na 2/3 (sekce 4.3.3). Konvence zřejmě nepůsobí jako přímé vynucení, ale spíš jako *aktivace* latentních znalostí modelu, tedy jako připomínka toho, co se model v tréninkových datech naučil. Toto rozlišení je interpretační pojmenování výsledků této studie, ne převzatá taxonomie z literatury. Z toho plyne praktické rozlišení: měřitelné požadavky převést na verifikovatelné kroky v pre-PR checklistu, zatímco požadavky, které nelze měřit deterministicky (srozumitelnost dokumentace, vhodnost dekompozice), strukturovat jako nápovědu, která modelu připomíná žádoucí směr. Účinnost deterministické větve přitom závisí na existenci nástroje s jednoznačným výstupem. Tam, kde takový nástroj chybí, je míra operacionalizace principiálně nižší (sekce 5.5). Závěr o designové kvalitě stojí na Q8, tedy na podpůrné judge-based metrice bez κ validace, a je proto nutné jej číst opatrně.

Počítat s nedeterminismem a měřit proces.

Agent dodržoval verifikační příkazy deterministicky, ale vícekový pracovní postup (issue → branch → test → PR) nedeterministicky: pilotní běh r3 dosáhl P1 (issues before code) = 5/5, běh r5 při téměř identických instrukcích P1 = ~1/5 (sekce 5.3). Na úrovni výsledku (Q2) přitom rozdíl nemusel být patrný. procesní metriky (P1–P8) zachytily problém, který produktové metriky (Q1–Q8) samy o sobě neodhalily. Doporučení: nespoléhat na jednorázové ověření výstupu, zavést měřitelná exit kritéria pokrývající proces i kvalitu a kontrolovat je opakovaně.

Iterovat na základě dat.

Přínos této práce nespočívá v konkrétních instrukcích, ale v postupu jejich návrhu: naměřit metriky, diagnostikovat příčinu selhání, provést cílenou úpravu instrukcí a změřit znovu (sekce 3.1.3). Tento cyklus lze přenést na jiný model nebo projekt jen po přizpůsobení metrik a prahů danému prostředí. Podmínkou je sada metrik pokrývajících více dimenzí (proces, kvalitu produktu, zdroje), protože jednodimenzionální měření (např. jen test pass rate) může zakrýt problém v jiné dimenzi (sekce 5.1).

Praktický důsledek je přímočarý: místo delšího a obecnějšího `AGENTS.md` je účelnější budovat kratší, měřitelný pracovní postup s jasnými verifikačními kroky a opakovaně ho ladit podle dat.

5.5 Náměty pro další výzkum

Případová studie otevřela několik otázek, které přesahují rozsah této práce.

Nedeterminismus více krokového pracovního postupu.

Agent dodržoval verifikační příkazy (jednokrokové akce) deterministicky, ale více krokový pracovní postup (issue → branch → test → implement → PR) nedeterministicky: r3 kompletně, r5 vůbec, při téměř identických instrukcích. Možná vysvětlení zahrnují ztrátu pozornosti v dlouhém kontextovém okně, stochastickou volbu mezi strukturovaným a nestrukturovaným vývojem z tréninku modelu a pozici instrukce v kontextu. Granulární ablace pracovního postupu (které kroky jsou kritické?) a opakované běhy se stejnými instrukcemi by tento fenomén pomohly lépe pochopit.

Kontinuita práce mezi agenty.

procesní metriky (P1–P8) měří transparentnost artefaktů (issues, commit zprávy, PR popisy). Otevřená otázka: stačí tato transparentnost k tomu, aby jiný agent (nebo člověk) navázal na rozpracovaný projekt? Testování handoff scénářů by ověřilo, zda procesní instrukce vytváří nejen pozorovatelný, ale i přenositelný pracovní kontext.

Automatizace iteračního postupu.

Fáze Diagnóza a Úprava jsou v této práci prováděny manuálně autorem. Frameworky jako PromptWizard (Agarwal et al., 2024) a Prompt Alchemy (Ye et al., 2025) ukazují, že cyklus analýzy selhání a návrhu úprav lze automatizovat. Kombinace automatizovaného prompt optimizéru s metrikami P/Q/E by umožnila rychlejší iteraci bez manuální diagnózy.

Ověření načtení instrukcí.

Pilotní data ukazují, že agent instrukce přečte (v transcriptu cituje jejich obsah), ale ne vždy je aktivuje jako řídicí smyčku běhu. Místo pracovního postupu z `AGENTS.md` použije vlastní postup odvozený ze specifikace. Ověřitelný krok na začátku běhu (agent musí explicitně potvrdit přečtení a aktivaci pracovního postupu před prvním zápisem kódu) by mohl tento problém adresovat.

Mechanismy účinku instrukcí.

Naše data naznačují dva odlišné mechanismy, které v této práci označujeme jako *vynucení* (verifikační kroky přímo kontrolují výstup) a *aktivace* (obecné konvence připomínají modelu znalosti z tréninku). Analogie existuje v prompt engineeringu: chain-of-thought prompting ukazuje, že minimální nápověda může aktivovat komplexní latentní schopnost modelu efektivněji než detailní instrukce. Systematický experiment s odstupňovanou specificitou instrukcí, od

nápověd (“piš modulární kód”) přes pravidla (“každé pole musí být použito”) po verifikační kroky (“spust `tsc`, oprav chyby”), by mohl identifikovat, která míra konkrétnosti je pro který typ požadavku optimální. Otevřená je i otázka, zda se toto optimum posouvá se schopnostmi modelu. Lepší modely by mohly potřebovat méně specifické instrukce (analogicky k expertise reversal effect v kognitivní psychologii, kde instrukce účinné pro nováčky zpomalují experty).

Generalizovatelnost verifikačních kroků.

Účinnost verifikačních kroků v naší studii závisela na existenci deterministických nástrojů (`eslint`, `tsc`, test runner), které produkují jednoznačný výstup zpět do kontextu agenta. Kódování je v tomto ohledu specifická doména: řada požadavků na kvalitu má odpovídající nástroj. U jiných typů úloh (dokumentace, architektonická rozhodnutí, plánování) takové nástroje často neexistují a operacionalizace se může zastavit na úrovni pravidla nebo příkazu bez externí zpětné vazby. Otevřená otázka je, zda principy operacionalizace platí i mimo kódování, nebo zda je jejich účinnost podmíněna dostupností deterministické zpětné vazby.

Rozšíření na další modely, projekty a domény.

Výsledky platí pro jeden model (MiniMax-M2.5) a jeden typ projektu (deterministická business logika). První osa dalšího výzkumu je replikace uvnitř programování: jiné modely, jiné agentní nástroje a projekty s vyšší nejednoznačností specifikace (sekce 5.3). Druhá osa je ověření mimo programování, kde nemusí existovat ekvivalent test runneru, lintu nebo typechecku. Právě tam se ukáže, zda vzorec pravidlo → příkaz → verifikační krok vyjadřuje obecnější princip řízení agentů, nebo hlavně výhodu programovací domény s deterministickou zpětnou vazbou.

Exit kritéria jako součást instrukcí.

V naší studii zůstávala exit kritéria mimo `AGENTS.md`: agent dostal instrukce, *jak* pracovat, ne *jakých hodnot* metrik dosáhnout. Otevřenou otázkou je, zda by zahrnutí kritérií přímo do instrukcí, například jako pre-PR self-check (“spust `tsc --noEmit`, žádné chyby”), výsledky zlepšilo, nebo zda by agent kritéria začal optimalizovat triviálními zkratkami (verbose kód proti prahu složitosti, povrchní testy proti pokrytí). Riziko Goodhartova zákona je v úzké sadě metrik vyšší; sada pokrývající tři dimenze ho částečně omezuje, protože gaming jedné metriky se projeví v jiné. Hranice mezi legitimním řízením a změnou povahy měřeného chování zůstává otevřenou otázkou.

Závěr

Tato práce navrhla sadu metrik a iterativní postup pro systematické navrhování a vyhodnocování instrukcí pro AI coding agenty a ablacemi prozkoumala vliv jednotlivých složek instrukcí. Pro všechny tři cíle případová studie systému upomínek faktur demonstrovala proveditelnost navrženého postupu a přinesla indikativní poznatky. Studie zahrnovala pět pilotních iterací a dvě ablace, každou ověřenou dvěma nezávislými běhy.

Prvním cílem bylo navrhnout sadu metrik pokrývající proces, kvalitu produktu a zdroje, a tím rozšířit hodnocení nad rámec pass/fail výsledku. Výsledná sada 19 metrik pokrývá tři dimenze odvozené z taxonomie Fentona a Biemana. Procesní metriky odhalily problémy, které výsledkové metriky samy o sobě nezachytily: běh s téměř plným skóre funkčních testů vykazoval nestabilní procesní compliance. Kombinace deterministických a judge-based metrik přinesla komplementární pohled, protože každý typ zachycuje jinou dimenzi chování agenta.

Druhým cílem bylo na případové studii demonstrovat iterativní postup návrhu instrukcí řízený těmito metrikami a vyhodnotit, zda a jak vede k měřitelným změnám v chování agenta. Cyklus měření, diagnózy a úpravy vedl k měřitelnému zlepšení, ale ne monotónnímu: jedna iterace způsobila regresi, po které bylo nutné vrátit se k předchozí verzi instrukcí. Opakujícím se vzorcem úspěšných úprav bylo nahrazení obecného pravidla konkrétním příkazem s verifikačním krokem. Účinnost tohoto postupu závisela na dostupnosti deterministických nástrojů s jednoznačným výstupem. U požadavků bez takového nástroje zůstává instrukce na úrovni pravidla nebo příkazu bez vnější zpětné vazby.

Třetím cílem bylo ablacemi prozkoumat, které složky instrukcí vytvořených v cíli 2 přispívají k měřenému chování agenta a které jsou redundantní. Ablace potvrdila závislost klíčových metrik na verifikačních krocích, kterou pilotní fáze už naznačila: jejich odebrání zhoršilo funkční korektnost i čistotu kódu. Konvence kvality kódu automatizované metriky neovlivnily, ale designová kvalita hodnocená LLM-as-judge klesla. Výsledky zároveň ukázaly, že procesní compliance je výrazně citlivá na nedeterminismus modelu. Stejná sada instrukcí proto může vést k odlišnému dodržení vícekrokového pracovního postupu.

Hlavním přínosem práce je iterativní postup a sada metrik, které může praktik aplikovat na vlastním projektu s vlastními prahy. Konkrétní instrukce a naměřené hodnoty platí pro tuto případovou studii. Přenositelný je postup, ne výsledná čísla. Práce tím demonstruje proveditelnost navrženého rámce na jednom případě, zatímco závěry mají povahu analytické generalizace: formulují principy k dalšímu ověření, zejména otázku platnosti vzorce operacionalizace mimo programování, nikoliv statistické tvrzení o populaci.

Pro praktika z toho plyne jednoduché doporučení: nestačí sledovat, zda agent úkol vyřešil, ale i jak k řešení došel a jak kvalitní kód zanechal. Pro akademického čtenáře práce ukazuje, že instrukce lze studovat ne jen jako promptový artefakt, ale jako samostatně měřenou nezávislou proměnnou, jejíž účinek je třeba hodnotit přes proces, produkt i náklady.

Literatura

- ACE-Bench: End-to-End Feature Development Benchmark [212 tasks from 16 repos; Claude 4 Sonnet + OpenHands: 7.5% on feature-level tasks]. (2025). <https://openreview.net/forum?id=41xrZ3uGuI>
- Agarwal, E., Dani, V., Ganu, T., & Nambi, A. (2024). PromptWizard: Task-Aware Prompt Optimization Framework. *arXiv preprint arXiv:2405.18369*. <https://arxiv.org/abs/2405.18369>
- Ammann, P., & Offutt, J. (2016). *Introduction to Software Testing* (2nd ed.). Cambridge University Press.
- Bacchelli, A., & Bird, C. (2013). Expectations, Outcomes, and Challenges of Modern Code Review. *Proceedings of the 35th International Conference on Software Engineering*, 712–721. <https://doi.org/10.1109/ICSE.2013.6606617>
- Beck, K. (2000). *Extreme Programming Explained: Embrace Change*. Addison-Wesley.
- Beller, M., Bholanath, R., McIntosh, S., & Zaidman, A. (2016). Analyzing the State of Static Analysis: A Large-Scale Evaluation in Open Source Software. *IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, 470–481. <https://doi.org/10.1109/SANER.2016.105>
- Beller, M., Gousios, G., Panichella, A., Proksch, S., Amann, S., & Zaidman, A. (2019). Developer Testing in the IDE: Patterns, Beliefs, and Behavior [Large-scale field study (2,443 engineers, 2.5 years, four IDEs) of how developers actually test. Findings: half do not test, TDD rarely practiced, test effort much lower than self-reported. Supports test-related practices as observable behavior distinct from stated ideals.]. *IEEE Transactions on Software Engineering*, 45(3), 261–284. <https://doi.org/10.1109/TSE.2017.2776152>
- Bissyandé, T. F., Lo, D., Jiang, L., Réveillère, L., Klein, J., & Le Traon, Y. (2013). Got Issues? Who Cares About It? A Large Scale Investigation of Issue Trackers from GitHub. *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, 188–197. <https://doi.org/10.1109/ISSRE.2013.6698918>
- Boehm, B. W. (1981). *Software Engineering Economics* [Seminal work on software cost estimation; introduced the COCOMO (Constructive Cost Model) based on project size (SLOC) and effort multipliers.]. Prentice-Hall.
- Boehm, B. W., Abts, C., Brown, A. W., Chulani, S., Clark, B. K., Horowitz, E., Madachy, R., Reifer, D. J., & Steece, B. (2000). *Software Cost Estimation with COCOMO II*. Prentice Hall.
- Breunig, D. (2025). Don't Fight the Weights [Defines fighting-the-weights: when prompt instructions oppose trained model behavior; recognition signs and mitigation strategies]. <https://www.dbreunig.com/2025/11/11/don-t-fight-the-weights.html>
- Brooks, F. P. (1987). No Silver Bullet: Essence and Accidents of Software Engineering. *Computer*, 20(4), 10–19. <https://doi.org/10.1109/MC.1987.1663532>
- Ehsani, R., et al. (2026). Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub [33k agent PRs, 5 coding agents, taxonomy of failures]. *Proceedings of MSR 2026*. <https://arxiv.org/abs/2601.15195>

- FeatureBench: Benchmarking Agentic Coding for Complex Feature Development [Agents struggle when scope exceeds single-issue; feature-level tasks significantly harder]. (2026). *arXiv preprint arXiv:2602.10975*.
- Fenton, N. E., & Bieman, J. (2014). *Software Metrics: A Rigorous and Practical Approach* (3rd ed.) [Process vs. product vs. resource metrics taxonomy; GQM framework]. CRC Press.
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
- Forsgren, N., Storey, M.-A., Maddila, C., Zimmermann, T., Houck, B., & Butler, J. (2021). The SPACE of Developer Productivity: There’s More to It than You Think [Proposes the SPACE framework (Satisfaction, Performance, Activity, Communication, Efficiency) for individual developer productivity. Explicit warning against reducing productivity to a single metric (SLOC, commit counts). Conceptual precedent for multi-dimensional measurement used in this thesis.]. *Communications of the ACM*, 64(11), 46–53. <https://doi.org/10.1145/3453120>
- Gloaguen, T., Mündler, N., Müller, M., Raychev, V., & Vechev, M. (2026). Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? [LLM-generated context files reduce success rate; developer-written marginally help (+4%); context files increase cost 20%+; agents follow instructions but no performance gain; AGENTBENCH benchmark (138 instances, 12 repos)]. *arXiv preprint arXiv:2602.11988*. <https://arxiv.org/abs/2602.11988>
- Gotel, O. C., & Finkelstein, A. C. (1994). An Analysis of the Requirements Traceability Problem. *Proceedings of IEEE International Conference on Requirements Engineering*, 94–101.
- Guo, J., Huang, S., Li, M., Huang, D., Chen, X., Zhang, R., Guo, Z., Yu, H., Yiu, S.-M., Lio, P., & Lam, K.-Y. (2025). A Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System [Over 150 papers surveyed]. *arXiv preprint arXiv:2510.09721*. <https://arxiv.org/abs/2510.09721>
- Hassan, A. E., Li, H., Lin, D., Adams, B., Chen, T.-H., Kashiwa, Y., & Qiu, D. (2025). Agentic Software Engineering: Foundational Pillars and a Research Roadmap [SASE framework, BriefingScript, MentorScript, SE4H/SE4A duality]. *arXiv preprint arXiv:2509.06216*. <https://arxiv.org/abs/2509.06216>
- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., ... Zaremba, W. (2021). Evaluating Large Language Models Trained on Code [Introduces HumanEval benchmark (164 hand-written Python problems) and Codex model. pass@k metric for functional correctness.]. *arXiv preprint arXiv:2107.03374*. <https://arxiv.org/abs/2107.03374>
- IEEE Computer Society. (2024). *Guide to the Software Engineering Body of Knowledge* (Version 4.0). <https://www.computer.org/education/bodies-of-knowledge/software-engineering>

- Inozemtseva, L., & Holmes, R. (2014). Coverage Is Not Strongly Correlated with Test Suite Effectiveness. *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, 435–445. <https://doi.org/10.1145/2568225.2568271>
- ISO/IEC. (2023). *Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model* (tech. rep. No. ISO/IEC 25010:2023). International Organization for Standardization. Geneva. <https://www.iso.org/standard/78176.html>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? [2294 tasks from real GitHub repositories, de facto benchmark for AI coding agents]. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2310.06770>
- Jin, H., Huang, L., Cai, H., Yan, J., Li, B., & Chen, H. (2024). From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future [Six SE areas covered: requirement engineering, code generation, autonomous decision-making, software design, test generation, software maintenance.]. *arXiv preprint arXiv:2408.02479*. <https://arxiv.org/abs/2408.02479>
- Jørgensen, M., & Shepperd, M. (2007). A Systematic Review of Software Development Cost Estimation Studies. *IEEE Transactions on Software Engineering*, 33(1), 33–53. <https://doi.org/10.1109/TSE.2007.256943>
- Kim, O. (2025). DETAIL Matters: Measuring the Impact of Prompt Specificity on Reasoning in Large Language Models [Emory University. Framework pro evaluaci vlivu specifčnosti promptu na reasoning. 30 úloh, GPT-4 a O3-mini]. *arXiv preprint arXiv:2512.02246*. <https://arxiv.org/abs/2512.02246>
- Kitchenham, B., & Pfleeger, S. L. (1996). Software Quality: The Elusive Target. *IEEE Software*, 13(1), 12–21. <https://doi.org/10.1109/52.476281>
- Lehman, M. M. (1980). Programs, Life Cycles, and Laws of Software Evolution. *Proceedings of the IEEE*, 68(9), 1060–1076. <https://doi.org/10.1109/PROC.1980.11805>
- Li, B., Latendresse, J., Abdalkareem, R., & Shang, W. (2026). Beyond Bug Fixes: Post-Merge Code Quality Issues in Agent-Generated Pull Requests [1210 merged agent PRs, SonarQube analysis. Merge success unreliable indicator of quality. Code smells, complexity, duplicated code persist.]. *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*. <https://arxiv.org/abs/2601.20109>
- Li, X., Chen, W., Liu, Y., Zheng, S., Chen, X., He, Y., Li, Y., et al. (2026). SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks [Curated Skills +16.2pp; self-generated Skills -1.3pp; 2–3 focused modules optimal; comprehensive documentation hurts (-2.9pp); smaller model + Skills can exceed larger model without; 86 tasks, 11 domains, 7308 trajectories]. *arXiv preprint arXiv:2602.12670*. <https://arxiv.org/abs/2602.12670>
- Lindenbauer, T., et al. (2025). The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management [Observation masking halves cost, matches LLM summarization; hybrid reduces cost 7–11% further]. *NeurIPS*.

-
- Liu, J., Wang, K., Chen, Y., Peng, X., Chen, Z., Zhang, L., & Lou, Y. (2024). Large Language Model-Based Agents for Software Engineering: A Survey [124 papers surveyed]. *arXiv preprint arXiv:2409.02977*. <https://arxiv.org/abs/2409.02977>
- Lulla, J. L., Mohsenimofidi, S., Galster, M., Zhang, J. M., Baltes, S., & Treude, C. (2026). On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents [124 PRs across 10 repos: -28% runtime, -20% output tokens with AGENTS.md; architectural info and coding conventions most effective]. *arXiv preprint arXiv:2601.20404*. <https://arxiv.org/abs/2601.20404>
- Mao, Y., He, J., & Chen, C. (2025). From Prompts to Templates: A Systematic Prompt Template Analysis for Real-world LLM Applications [Merged 4 frameworks (Google Cloud, Elavis Saravia, CRISPE, LangGPT); 7 component types; Directive 87% prevalence]. *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. <https://arxiv.org/abs/2504.02052>
- Mathews, N. S., & Nagappan, M. (2024). LLM-Based Test Generation as Bug Validation: Insights from Reproducing Real-World Bugs [68.1% generated test suites validate bugs instead of detecting them]. *Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware)*. <https://doi.org/10.1145/3664646.3664766>
- McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering, SE-2*(4), 308–320. <https://doi.org/10.1109/TSE.1976.233837>
- McConnell, S. (2004). *Code Complete: A Practical Handbook of Software Construction* (2nd). Microsoft Press.
- McIntosh, S., Kamei, Y., Adams, B., & Hassan, A. E. (2016). An Empirical Study of the Impact of Modern Code Review Practices on Software Quality. *Empirical Software Engineering, 21*(5), 2146–2189. <https://doi.org/10.1007/s10664-015-9381-9>
- Merrill, M. A., et al. (2026). Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces [89 hard CLI tasks; Stanford and Laude Institute; frontier models below 65 %]. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2601.11868>
- METR. (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity [RCT: 16 developers, 246 tasks; AI tools slowed experienced developers by 19%; developers believed they were 24% faster]. *arXiv preprint arXiv:2507.09089*. <https://arxiv.org/abs/2507.09089>
- METR. (2026). Many SWE-bench-Passing PRs Would Not Be Merged into Main [296 patches reviewed by 4 maintainers from 3 SWE-bench repos. ~24pp gap between automated pass rate and maintainer merge decisions. Technical report.]. <https://metr.org/notes/2026-03-10-many-swe-bench-passing-prs-would-not-be-merged-into-main/>
- Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., & Zettlemoyer, L. (2022). Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? [Label space, input distribution a formát jsou klíčové pro ICL, ne správnost label-input mapování]. *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://arxiv.org/abs/2202.12837>
- Nonaka, I., & Takeuchi, H. (1995). *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press.
-

- Panickssery, A., Bowman, S. R., & Feng, S. (2024). LLM Evaluators Recognize and Favor Their Own Generations [Causal link: self-recognition $\rightarrow$ self-preference; linear correlation; fine-tuning pushes recognition to 90%+]. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2404.13076>
- Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., & Harman, M. (2019). Mutation Testing Advances: An Analysis and Survey [Mutation score stronger predictor of fault detection than structural coverage]. *Advances in Computers*, 112, 275–378. <https://doi.org/10.1016/bs.adcom.2018.03.015>
- Peffer, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2008). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>
- Rafique, Y., & Mišić, V. B. (2013). The Effects of Test-Driven Development on External Quality and Productivity: A Meta-Analysis. *IEEE Transactions on Software Engineering*, 39(6), 835–856. <https://doi.org/10.1109/TSE.2012.28>
- Razavi, S. P., & Fard, F. H. (2025). What Did I Do Wrong? Quantifying LLMs’ Sensitivity and Consistency to Prompt Engineering [Up to 76 accuracy points difference from subtle formatting changes in few-shot settings]. *Proceedings of NAACL*. <https://arxiv.org/abs/2406.12334>
- Runeson, P., & Höst, M. (2009). Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- Scale AI. (2025). SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? [1865 human-verified tasks, adds explicit Requirements and Interface sections to issues]. *arXiv preprint arXiv:2509.16941*. <https://arxiv.org/abs/2509.16941>
- Shin, J., Tang, C., Mohati, T., Nayebi, M., Wang, S., & Hemmati, H. (2025). Prompt Engineering or Fine-Tuning: An Empirical Assessment of Large Language Models for Code. *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR)*. <https://arxiv.org/abs/2310.10508>
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761*.
- Schnabel, T., & Neville, J. (2024). Symbolic Prompt Program Search: A Structure-Aware Approach to Efficient Compile-Time Prompt Optimization. *arXiv preprint arXiv:2404.02319*. <https://arxiv.org/abs/2404.02319>
- Solis, C., & Wang, X. (2011). A Study of the Characteristics of Behaviour Driven Development. *37th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA)*, 383–387. <https://doi.org/10.1109/SEAA.2011.76>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
- Verga, P., Hofstätter, S., Cer, D., & Thorne, J. (2024). Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models [Panel of LLM evaluators (PoLL)]

- outperforms single GPT-4 judge; disjoint model families reduce intra-model bias; 7x cheaper]. *arXiv preprint arXiv:2404.18796*. <https://arxiv.org/abs/2404.18796>
- Watanabe, M., Li, H., Kashiwa, Y., Reid, B., Iida, H., & Hassan, A. E. (2025). On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub [Under review. 567 Claude Code PRs, 157 projects, 83.8% acceptance rate]. *ACM Transactions on Software Engineering and Methodology*.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [Google Research. Few-shot CoT exempláře zlepšují reasoning na aritmetických, commonsense a symbolických úlohách]. *Advances in Neural Information Processing Systems (NeurIPS)*, 35. <https://arxiv.org/abs/2201.11903>
- Yang, J., Jimenez, C. E., Wettig, A., Liber, K., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering [ACI ablation: +10.7pp over baseline shell; context management and error guardrails largest gains]. *NeurIPS*.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.
- Ye, S., Sun, Z., Wang, G., Guo, L., Liang, Q., Li, Z., & Liu, Y. (2025). Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation. *arXiv preprint arXiv:2503.11085*. <https://arxiv.org/abs/2503.11085>
- Yin, R. K. (2018). *Case Study Research and Applications: Design and Methods* (6th ed.) [Embedded single-case design; analytic generalization (to theory, not population)]. SAGE Publications.
- Yin, Z., Wang, J., Li, J., Shi, D., Wei, Y., Yue, Y., Liu, Z., Xia, X., & Lo, D. (2025). A Comprehensive Empirical Evaluation of Agent Frameworks on Code-centric Software Engineering Tasks. *arXiv preprint arXiv:2511.00872*.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena [Foundational LLM-as-judge paper; position bias, verbosity bias, self-enhancement bias; GPT-4 >80% human agreement]. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2306.05685>
- Zi, Y., Menon, H., & Guha, A. (2025). More Than a Score: Probing the Impact of Prompt Specificity on LLM Code Generation [PartialOrderEval framework. Partial order of prompts from minimal to maximally detailed. HumanEval + ParEval]. *arXiv preprint arXiv:2508.03678*. <https://arxiv.org/abs/2508.03678>

Přílohy

A. Využití nástrojů umělé inteligence

Při přípravě práce byl využit nástroj Claude Code (Anthropic) jako AI asistent v následujících oblastech:

Psaní textu práce. Nástroj byl využit pro drafting a úpravu textových částí (kapitoly 1–5), kontrolu konzistence argumentace a LaTeX formátování. Autor formuloval záměr a strukturu každé sekce; AI asistent navrhoval formulace, které autor revidoval a upravoval. Metodika, argumentace a finální znění jsou autorovy vlastní.

Vyhledávání a shrnutí literatury. Nástroj byl využit k identifikaci relevantních zdrojů a shrnutí obsahu zdrojů. Autor ověřoval relevanci a správnost každého zdroje před zařazením do textu; citace odkazují výhradně na zdroje přečtené a posouzené autorem.

Diagnostická analýza ve studii. V kroku Diagnóza iteračního cyklu (sekce 3.1.3) byl nástroj využit jako analytický asistent: výzkumník popisoval naměřené výsledky, AI asistent navrhoval možné příčiny selhání a opravy instrukcí na základě literatury. Finální rozhodnutí o každé změně `AGENTS.md` bylo vždy na autorovi.

Implementace měřicí infrastruktury. Experimentální skripty (`new-run.ts`, `analyze-run.ts`, `judge.ts`) byly implementovány s asistencí AI nástroje na základě autorova návrhu specifikace metrik. Autor navrhl architekturu, definoval požadované výstupy a provedl validaci správnosti měření.

Claude Code nepouštěl experimentální běhy ani neprováděl finální interpretaci výsledků a nerozhodoval o směru výzkumu. Pro hodnocení části metrik (P6, P7, P8, Q4, Q8) byla ve studii samostatně použita metoda LLM-as-judge s modelem GLM-5; její role, omezení a ověřitelnost jsou popsány v sekci 3.2.4. Za celý obsah práce nese plnou odpovědnost autor.